

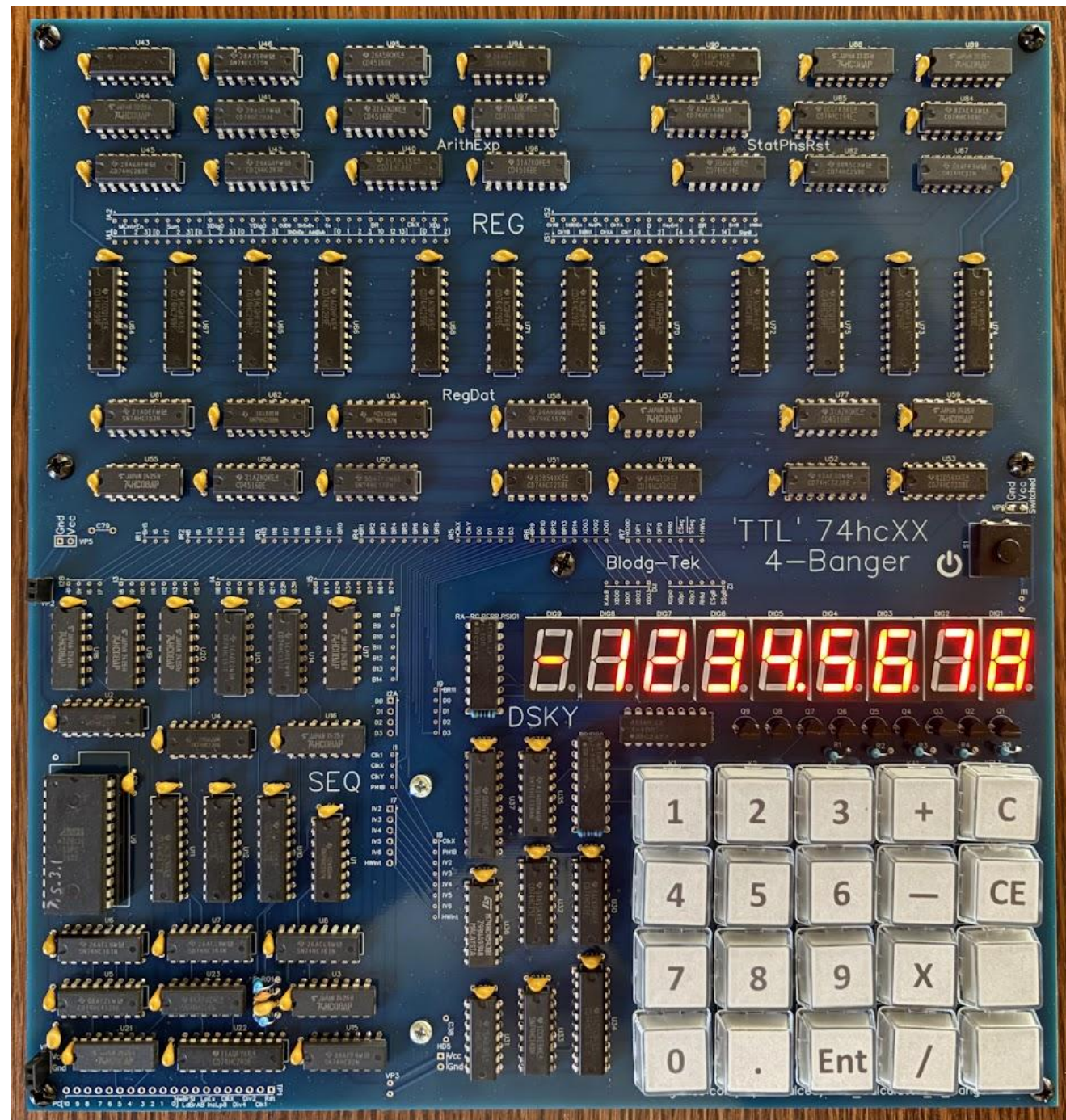
A "TTL" 4-Function Calculator Overview

Specs

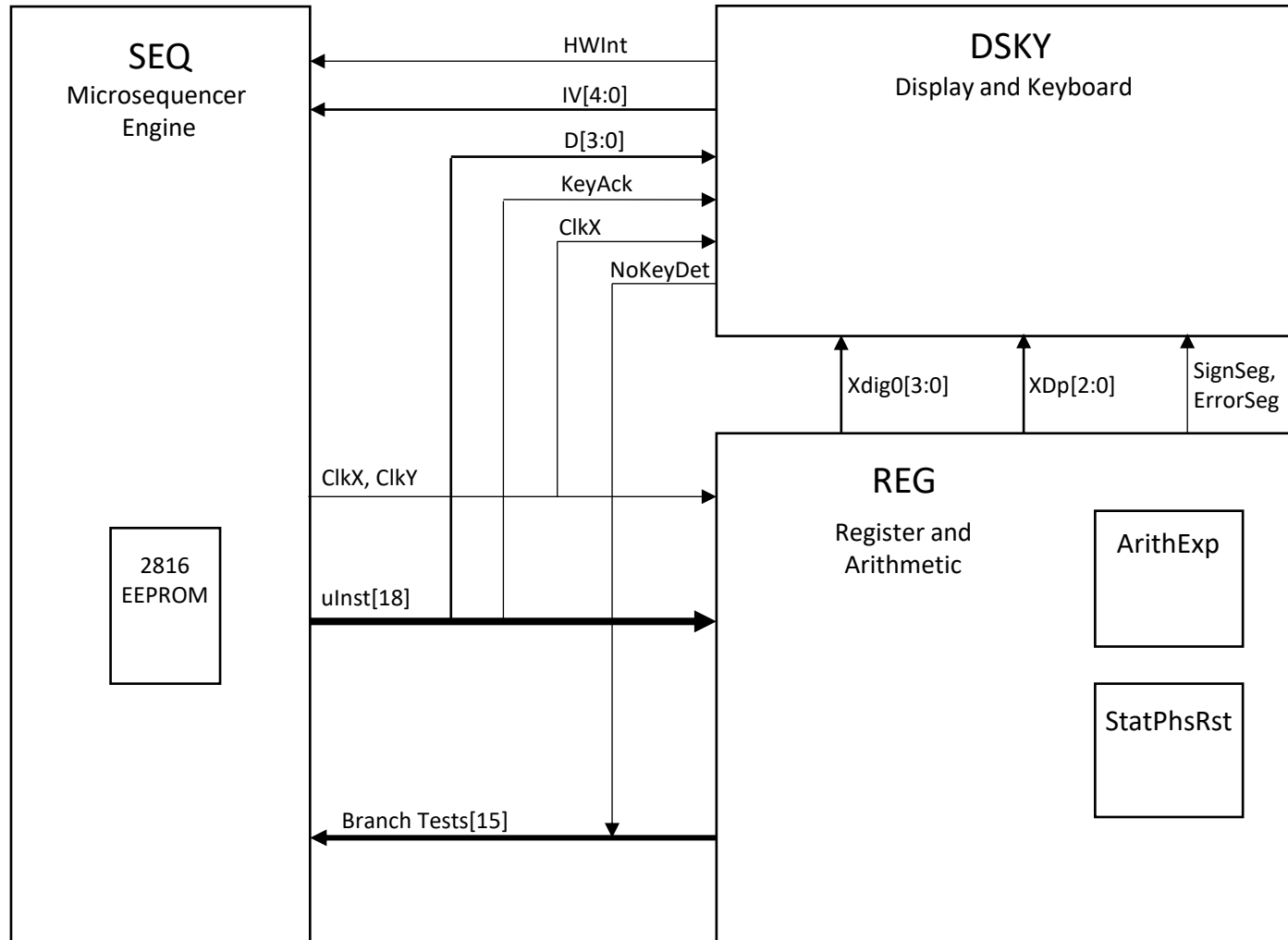
- 9x10" PCB, 4 layers
- All 74hcXX logic implementation (plus 3 4XXX) chips, 76 total, and one 2816 EEPROM
- Purpose built architecture (not a CPU architecture)
- RPN style operation (HP Enter key)
- Add, subtract, multiply, and divide, clear entry, clear all
- 8 numeric digits plus an Error and sign digit
 - Max num: +99,999,999 to -99,999,999
 - Min Num: +0.0000001 to -0.0000001
- Floating decimal point
- Signals error on overflow for all operations, divide by 0, and keyboard input overflow
- Leading zero blanking during operand entry
- Running total operation (last result = Operand1 + Enter)
- No rounding logic or guard digits

https://github.com/Sphaeralcea/TTL_Calculator_4_Banger

<https://www.youtube.com/watch?v=umok5LHqRb0>



Top Level



Overview

3 Major logic blocks

- SEQ: Micro-sequencer engine with 2Kx8b uROM
 - 24-bit micro-instruction word
 - Unconditional branch, 16 conditional branches, 1 NMI-type interrupt
 - Single level loop counter (no nesting)
 - Clock and reset generation
- DSKY: Keyboard and Display unit
 - 9-digit LED display sequencing,
 - 18(20)-key keyboard encoding (2 keys not used)
- REG: Register-Arithmetic unit
 - 3, 8-digit registers (4-bits X 8)
 - 1 digit BCD adder/subtractor and exponent calculator (counters)
 - 7 status and control register bits, operation progress tracking state machine

- Architectural deficits (intentional)
 - No rounding logic, guard digits
- Architectural bugs (unintentional)
 - Accuracy loss dividing 2 8-digit integers

Design Entry



Design
Entry

Schematic Entry

Analog Parts

unCode Entry

uCode Modeling

[illegible]

uCode Assembler Spreadsheet

[illegible]

LTSpice	
Disp	
Clk	
Rst	

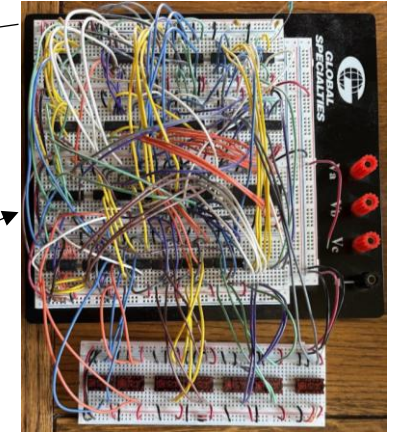
Txt File

Arduino Sketch

2816 Programmer

- Component placement is manual
- Autoroute used to route the modules
- Manual route to stitch together modules

BreadBoard

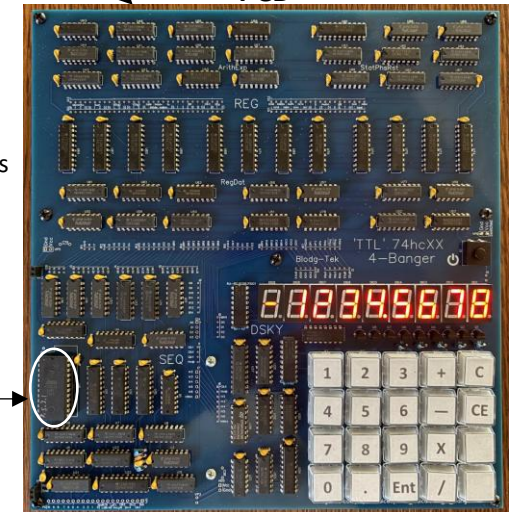


Design Feedback

EasyEDA std/JLCPCB

Manual Sch Translation ☹️
Manual Formal Verification ☹️

PCB



Microsequencer

Assembles 24-bit uInstruction from 3 consecutive reads of uROM.

4 Instruction Types:

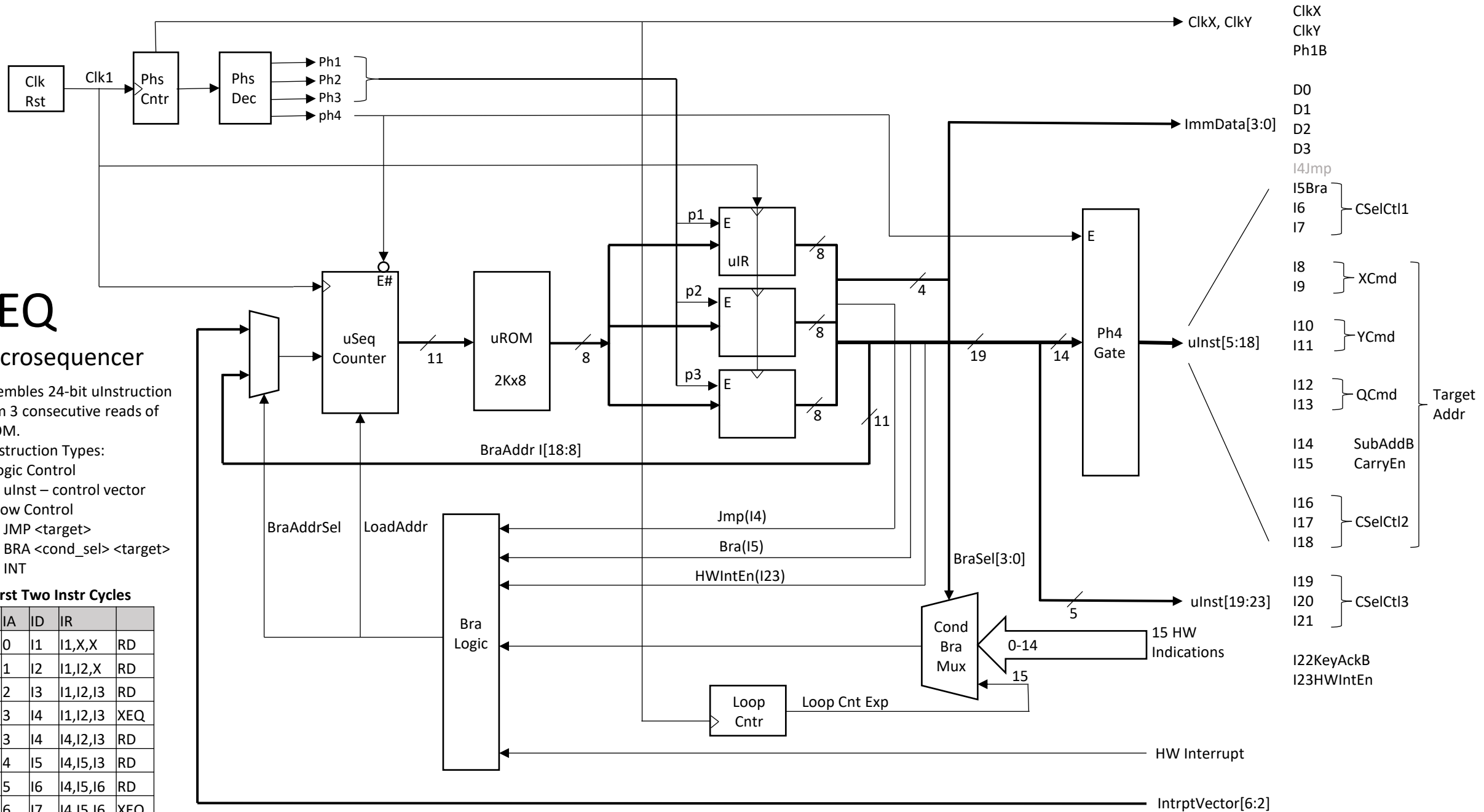
```

Logic Control
    ulnst – control vector
Flow Control
    JMP <target>
    BRA <cond_sel> <target>
    INT

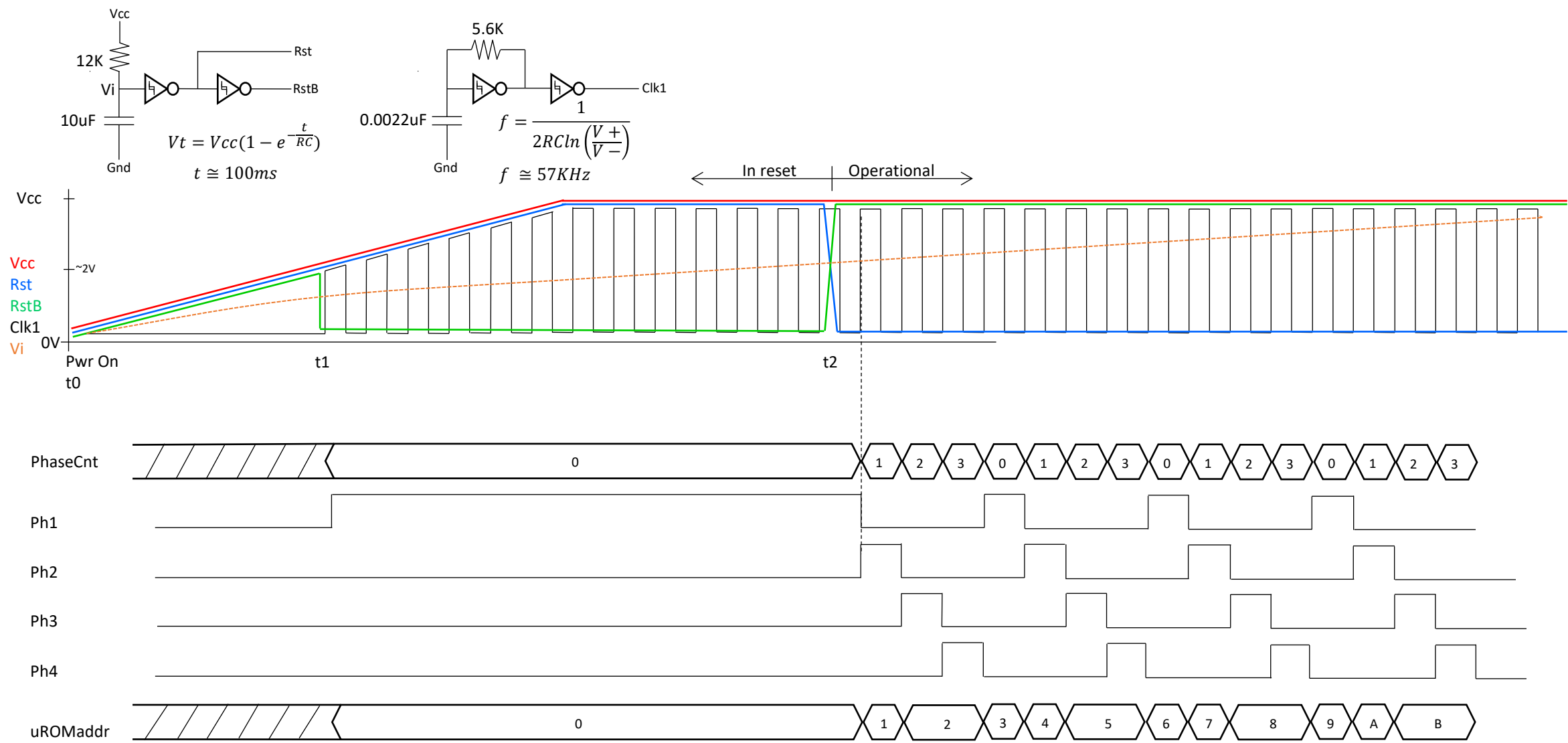
```

First Two Instr Cycles

Phs	IA	ID	IR	
P1	0	I1	I1,X,X	RD
P2	1	I2	I1,I2,X	RD
P3	2	I3	I1,I2,I3	RD
P4	3	I4	I1,I2,I3	XEQ
P1	3	I4	I4,I2,I3	RD
P2	4	I5	I4,I5,I3	RD
P3	5	I6	I4,I5,I6	RD
P4	6	I7	I4,I5,I6	XEQ



Power-On Reset Startup



Sequencer (SEQ)

- Includes Clock, Reset and Timing generation
 - Schmitt trigger reset holds sequencer in reset until power rises to $\sim 75\%$ VCC
 - Schmitt trigger oscillator generates 1X clock
 - 1X clock drives SEQ logic only. Does not clock other units
 - 1X clock feeds the uInst PC, the IR, and the phase counter which also generates the div4 clocks, ClkX and ClkY
 - ClkX and ClkY clock the other units
- Sequencer operates on a 4-phase instruction cycle to construct and present a 24-bit micro-instruction, 23 of which are available to control external hardware
 - First 3 phases read three consecutive bytes from uROM and stores them in 3 8-bit registers
 - During first 3 phases, external hardware sees 0's (inactive) on all signals
 - Fourth phase ungates the uInstruction and presents it to external hardware
 - D0 – D3: Immediate data and select field
 - I4: Instruction type. This bit is reserved for the sequencer's exclusive use
 - I4=0: Instruction is a hardware control instruction
 - I4=1: Instruction is a flow control instruction (JMP or BRA)

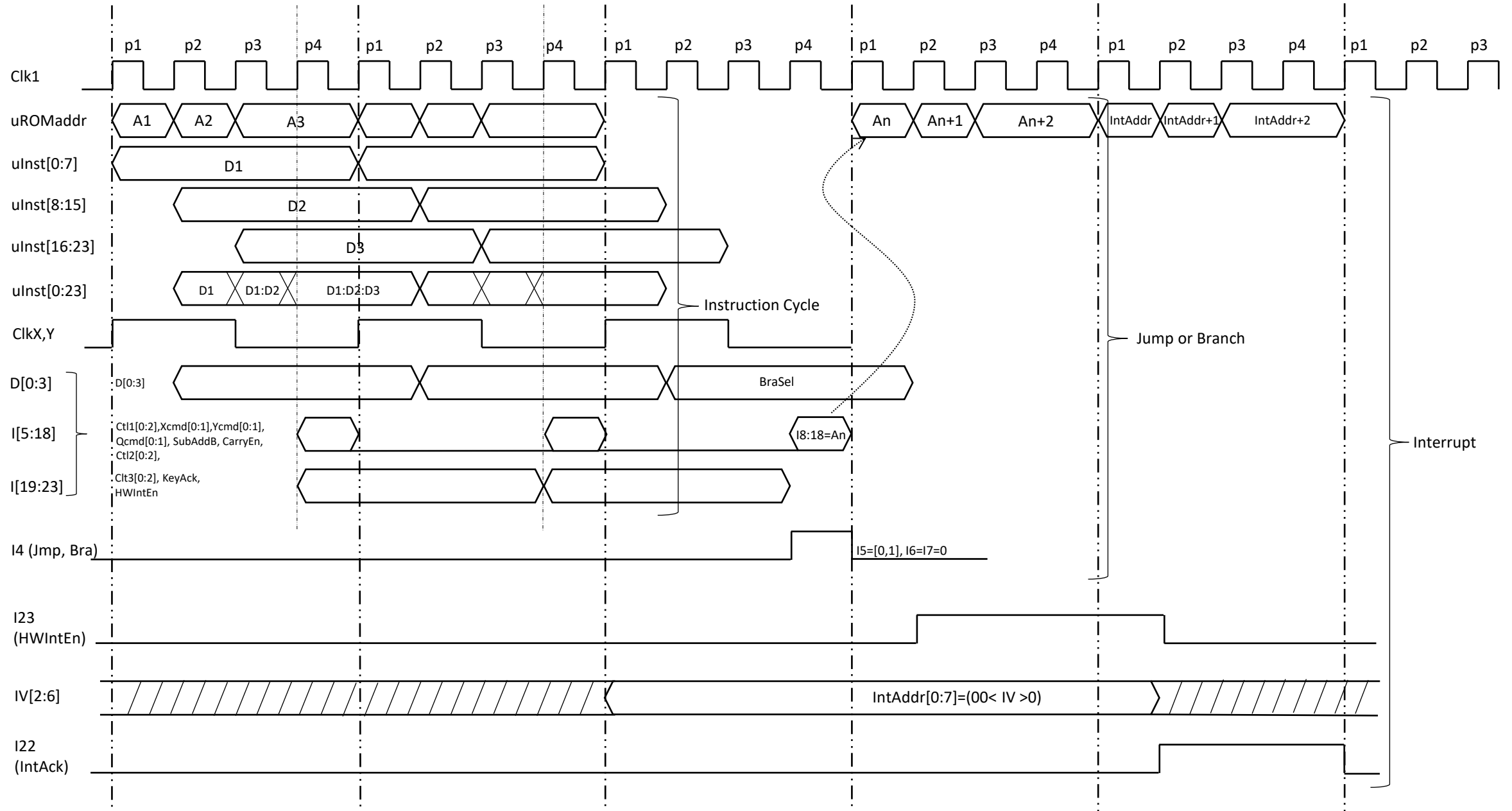
Sequencer (SEQ)

- I5: HW control and flow control qualifier
 - If I4=0, then I5 is just another hardware control bit
 - If I4=1, then I5 is flow control qualifier
 - I5=0, unconditional (JMP)
 - I5=1, conditional (BRA)
 - One of 16 conditions is tested, selected by D[3:0]. If condition is true then branch is taken
 - D[3:0]=0xF is reserved. Dedicated internally to implement a simple for-loop capability
- I6 – I21: uINSTR bits
- I22: Loop Init: Used internal to SEQ to reset a counter that implements a simple for-loop capability
 - I22 is also used as external to SEQ to register the state of keyboard (KeyAckB)
- I23: Interrupt enable. Used internal to SEQ to enable a hardware-initiated branch. Taken if external hardware has asserted the HWInt signal
 - External hardware provides the target address through a 5-bit vector, IV[6:2]
 - The target address is formed by appending IV[6:2] with b00
 - Hardware jump targets are confined to uROM addresses at 4-byte boundaries between 0x10 to 0x7C
 - I23 is available external to SEQ but not used

Sequencer (SEQ)

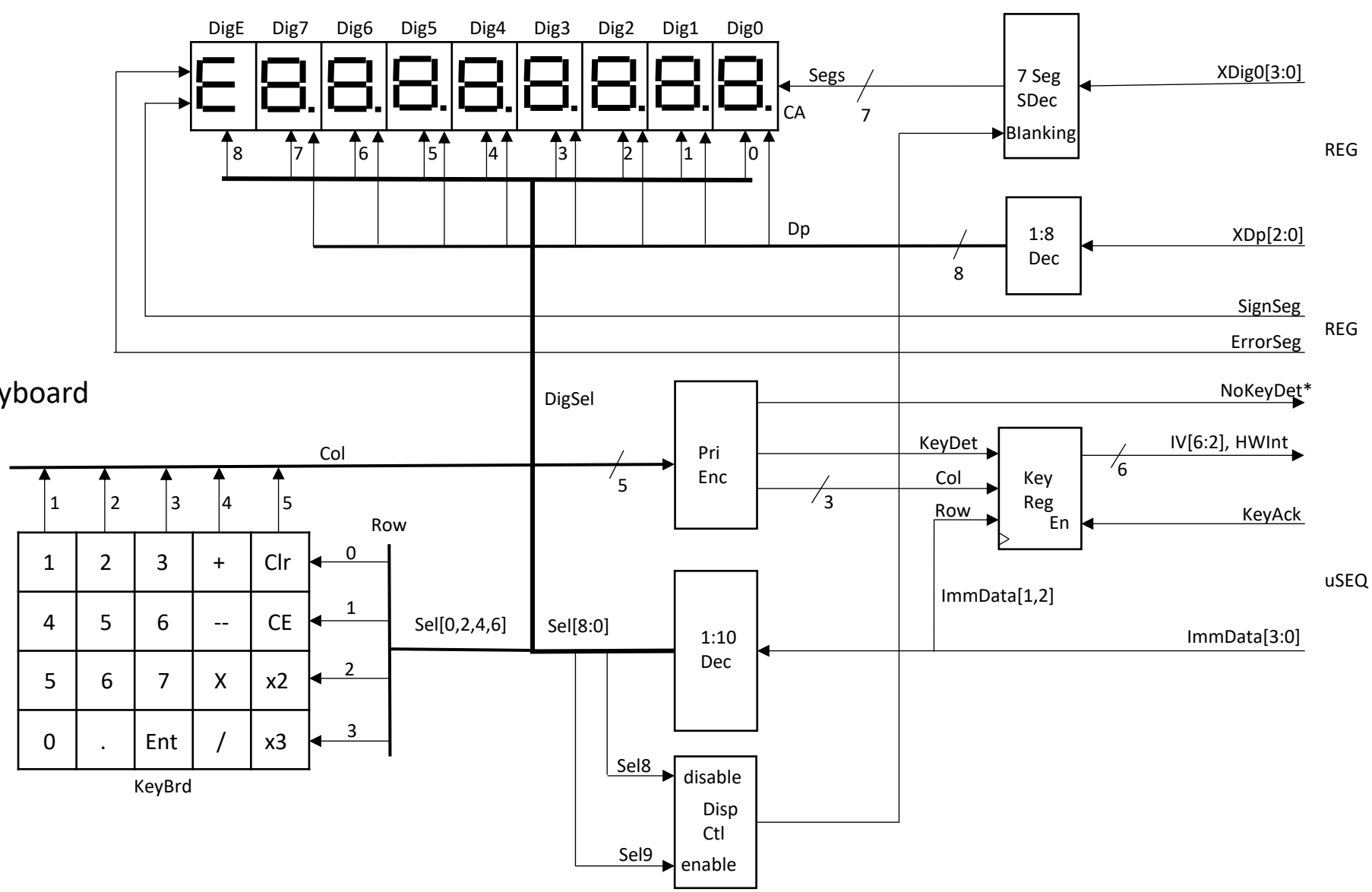
- A primitive looping structure provides the ability to create loops up to 9 iterations (0-8)
 - Loop counter reset by asserting I22 (InitLP8)
 - Loop count tested and incremented by a conditional jump with D[3:0]=0xF (BRLoop8NZ)
 - Loop count expires when loop count reaches 8, and loop will fall through on the next loop count test (BRLoop8NZ)
- Note: There is no subroutine call and return capability provided.
 - Since all keyboard service routines return to the keyboard-display loop, an unconditional JMP to this loop serves as an adequate return from keyboard service routines.

Sequencer Instruction Timing



DSKY

Display and Keyboard



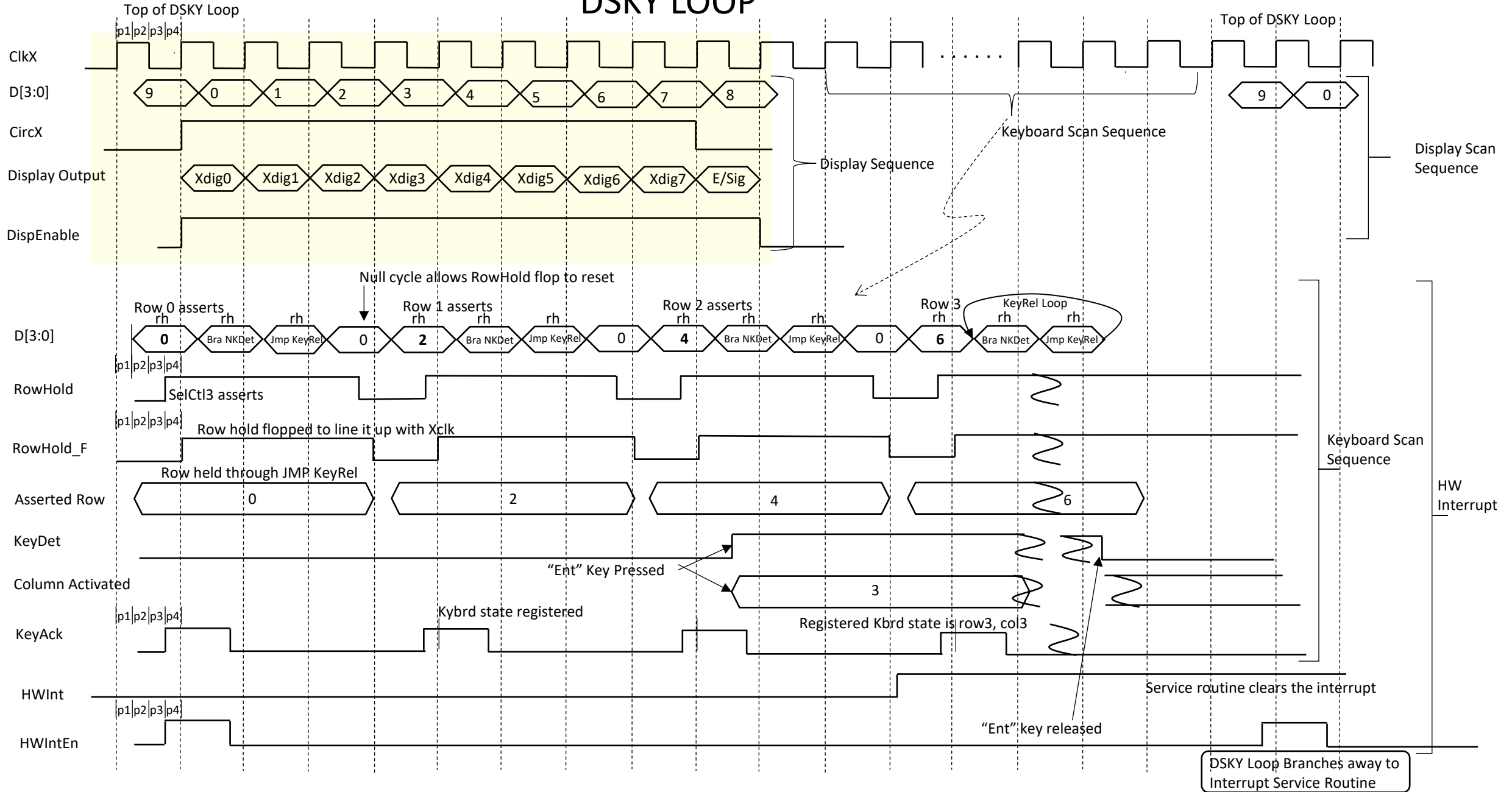
* Branch Condition

- Scan Sequence
- 9 – Disp En, IntEn
 - 0 – Dig0
 - 1 – Dig1
 - 2 – Dig2
 - 3 – Dig3
 - 4 – Dig4
 - 5 – Dig5
 - 6 – Dig6
 - 7 – Dig7
 - 8 – Dig8; Disp Dis
 - 0 – KeyRow1, KeyAck
 - 2 – KeyRow2, KeyAck
 - 4 – KeyRow3, KeyAck
 - 6 – KeyRow4, KeyAck

Display and Keyboard (DSKY)

- DSKY is a passive module with no autonomy. Completely controlled by SEQ
- Display and keyboard scan is performed via D[3:0] from SEQ
 - The display is enabled by D[3:0]=b1001, and it is disabled by b1000
 - Dig0 is activated by D[3:0]=b0000, Dig7 by b0111, the Sign/Error digit by b1000
 - Row 0 of the keyboard matrix is activated by b0000, row 1 by b0010, row 2 by 0100, and row 3 by b0110. The display is disabled during row scan
- REG provides BCD data for each digit to display, 3-bit decimal point position, and the sign and error segments
- When a key is pressed in an activated row, a key detect signal is raised
- If microcode running on SEQ tests the detect signal to be true, then it may assert the KeyAck (I22) signal back to DSKY which registers the row and column where the depressed key resides and asserts the HWInt signal
- On any microcode instruction that asserts HWIntEn (I23) while HWInt is asserted, SEQ flow will jump to the uROM address formed by the concatenation of col[2:0], row[1:0], and b'00

DSKY LOOP



REG: Registers, Data Steering and Compute

- REG is a passive unit with no autonomy. Completely controlled by SEQ
- Three Registers; XReg, YReg and QReg are each arrays of 8 BCD (4-bit) digits
 - Each register supports shift operations; either SHR, SHL, or both. SHR moves digits one position from MSD (most significant digit) to LSD, and the current LSD exiting the register with SRC replacing the MSD. SHL moves digits from LSD to MSD and the MSD exiting the register with SRC replacing the LSD
 - SHR SRC: X9, SUM, X_{LSD} , Y_{LSD}
 - SHL SRC: IMMData, Q_{MSD}
 - SHRY SRC: Zero, Y_{LSD} , X_{LSD}
 - SHRQ SRC: Zero, Y_{LSD}
 - SHLQ SRC: Zero, Q_{LSD}
 - Each register may operate individually or in tandem with others but only limited cases of simultaneous register shifting are supported. Examples include: Exchange the values of Xreg and Yreg; Adding Xreg and Yreg, with SUM replacing Xreg while Yreg circulates (or replaced with zero); Copy Xreg to Yreg while Xreg circulates (or replaced with zero)
- X9 and QL are single BCD digit registers (loadable counters), auxiliary to Xreg and Qreg, respectively.
 - X9 accumulates carries out of Xmsd for Addition and Multiplication operations and is used to test Xreg digits for 0
 - QL is used for Multiplication as a loop counter and for Division as partial quotient accumulator
 - Each can increment or decrement by one and each is loadable from two sources:
 - X9 SRC: Zero, X_{MSD}
 - QL SRC: Zero, Q_{LSD}

ArithExp – BCD Adder Subtractor

- ArithExp is a single digit BCD adder/subtractor that operates only on X_{LSD} and Y_{LSD} . The output of Arith is the SUM routed only to X_{MSD}
 - Additional outputs indicate if the add generated a carry out and if the result of a subtraction is a negative number.
 - Subtraction is done through 10s complement arithmetic.
- The Add/Sub block is implemented with a cascade of three combinational logic stages:
 - 10's complement of one input (subtraction)
 - Summing the inputs
 - BCD correction of result
- The CarryO bit is valid for exactly one clock following the sum. This imposes a requirement for any operation on multi-digit numbers be carried out with contiguous add instructions (no intervening cycles)
- The IsNeg bit is valid for exactly one clock following the sum. This requires that action to be taken as a result of the IsNeg signal is immediate.
- If the result of a subtraction is negative, the 10's complement result is stored in Xreg and additional uCode performs subtraction of the result from 0, returning the desired result

ArithExp – Metadata Counters

- ArithExp includes 4 counters that react to key strokes to compute sum of exponents, difference of exponents, digit count, and decimal point position
 - Incremented or decremented by microcode instructions
 - XDigCnt – increments for each numeric key. If zero(s) is the first keystroke, it is not counted
 - XDpCnt – increments for each numeric key after the decimal point (counts digits in fractional portion of operand)
 - SexpCnt, DexpCnt – hold the sum and difference, respectively, of the exponents of the two operands
 - Positive Exponent - first key of operand is numeric. Increment for each numeric key until decimal point is keyed
 - Negative Exponent - first key of operand is decimal point key. Decrement for each leading zero keyed until non zero numeric key
 - SexpCnt
 - for each operand, positive exponents increment up, negative exponents decrement down
 - DexpCnt
 - for first operand, positive exponents increment up, negative exponents decrement down
 - For second operand, negative exponents increment up, positive exponents decrement down
- Via microcode instructions, DexpCnt can be transferred to SexpCnt. SexpCnt can be transferred to XDpCnt. DexpCnt can be transferred to XDpCnt in two steps

StatPhsRst

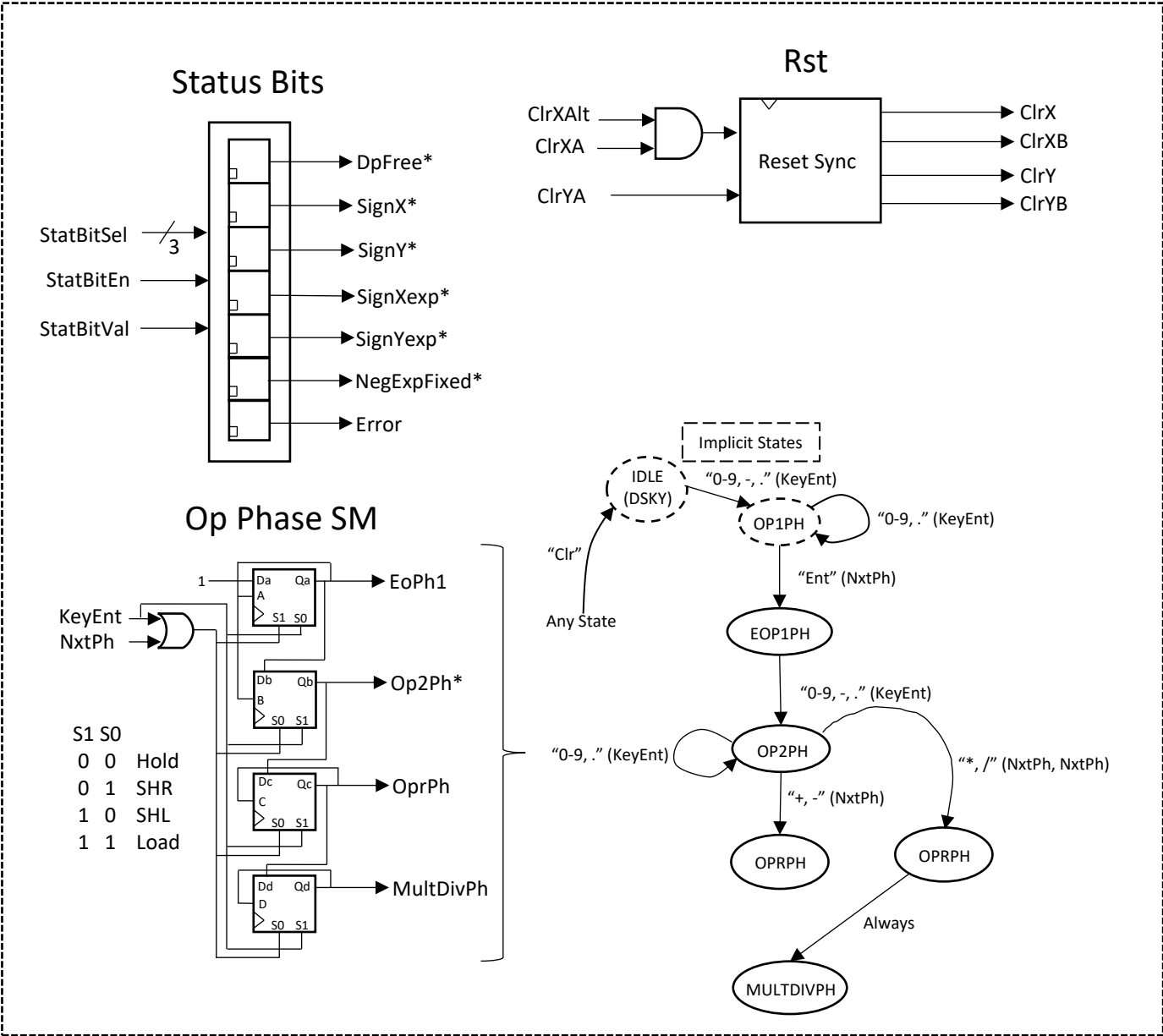
- There are two reset domains for sequential logic in REG, X and Y
 - The Xreg domain includes Xreg and state associated with Xreg
 - X reset is triggered by a microcode instruction and by the transition from entering the first operand to entering the second operand
 - The Yreg domain includes Yreg and Qreg and associated state
 - Y reset is triggered by a microcode instruction
 - The Rst part of StatPhsRst is a reset synchronizer to filter spurious logic glitches from SEQ on X and Y domain resets
 - It also includes single pulse generator that generates an alternate X domain reset when the first key of the second operand is detected
- The Stat part of StatPhsRst is an array of status bits to store sign bits for operands and exponents and other state information needed to parse keystrokes. Bits are set and cleared by microcode instructions and by reset events

StatPhsRst

- The Phs part of StatPhsRst is a state machine that tracks the phase of the operation underway. Its state variables are tested by branch instructions
- States include
 - Idle* – System has cycle through reset but no keys have been pressed (virtual state)
 - Op1Ph* – First operand is being entered (virtual state)
 - EOp1Ph – First operand is entered but second operand has not started
 - Op2Ph – Second operand is being entered
 - OprPh – One of the 4 operations is underway
- * Idle state and Op1Ph state do not need actual state variables but they represent operational states of the overall machine
- The state machine is advanced by a microcode instruction
 - The transition from EndOp1 to Op2Ph will also happen with the first key starting operand2

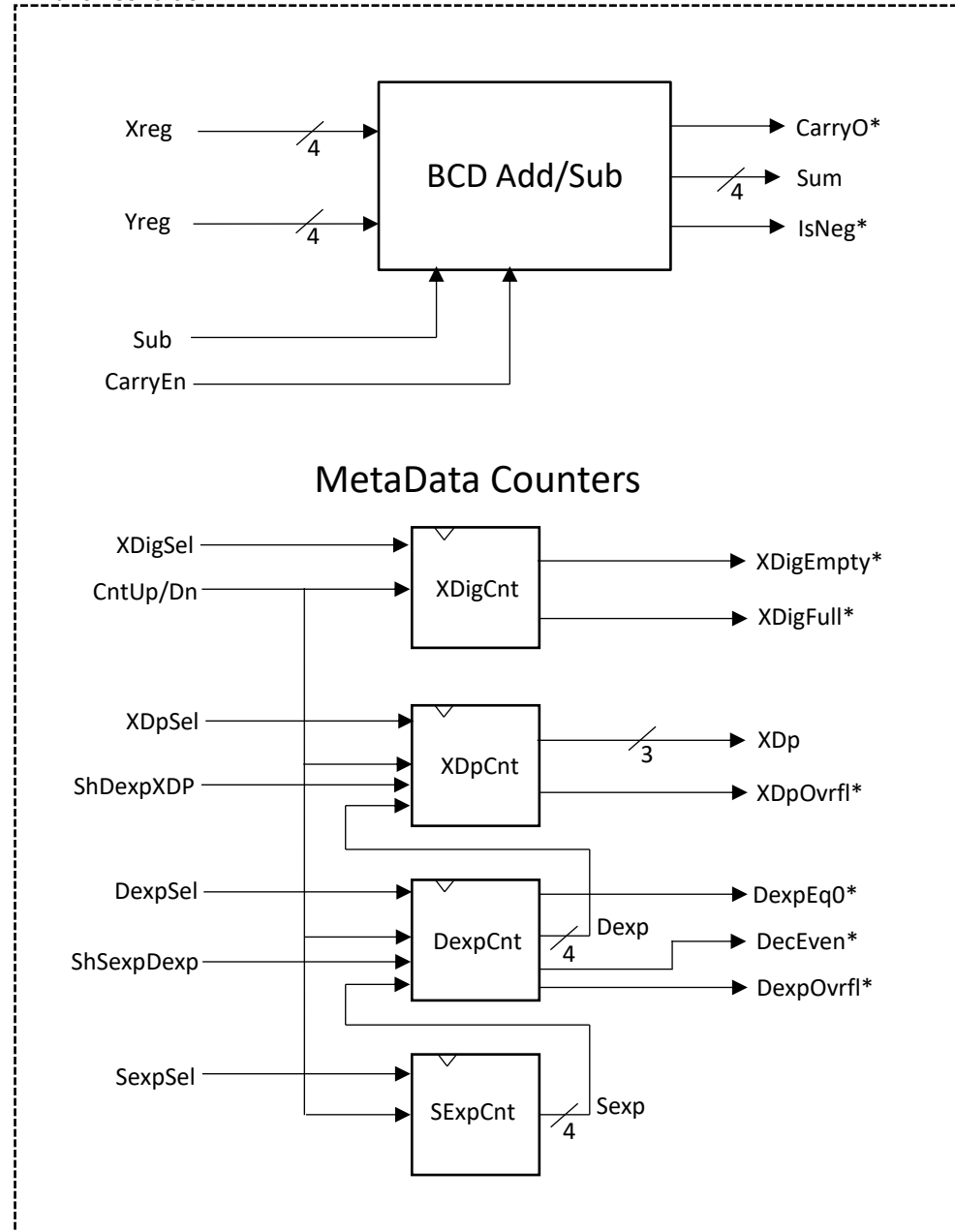
StatPhsRst

* Branch Condition

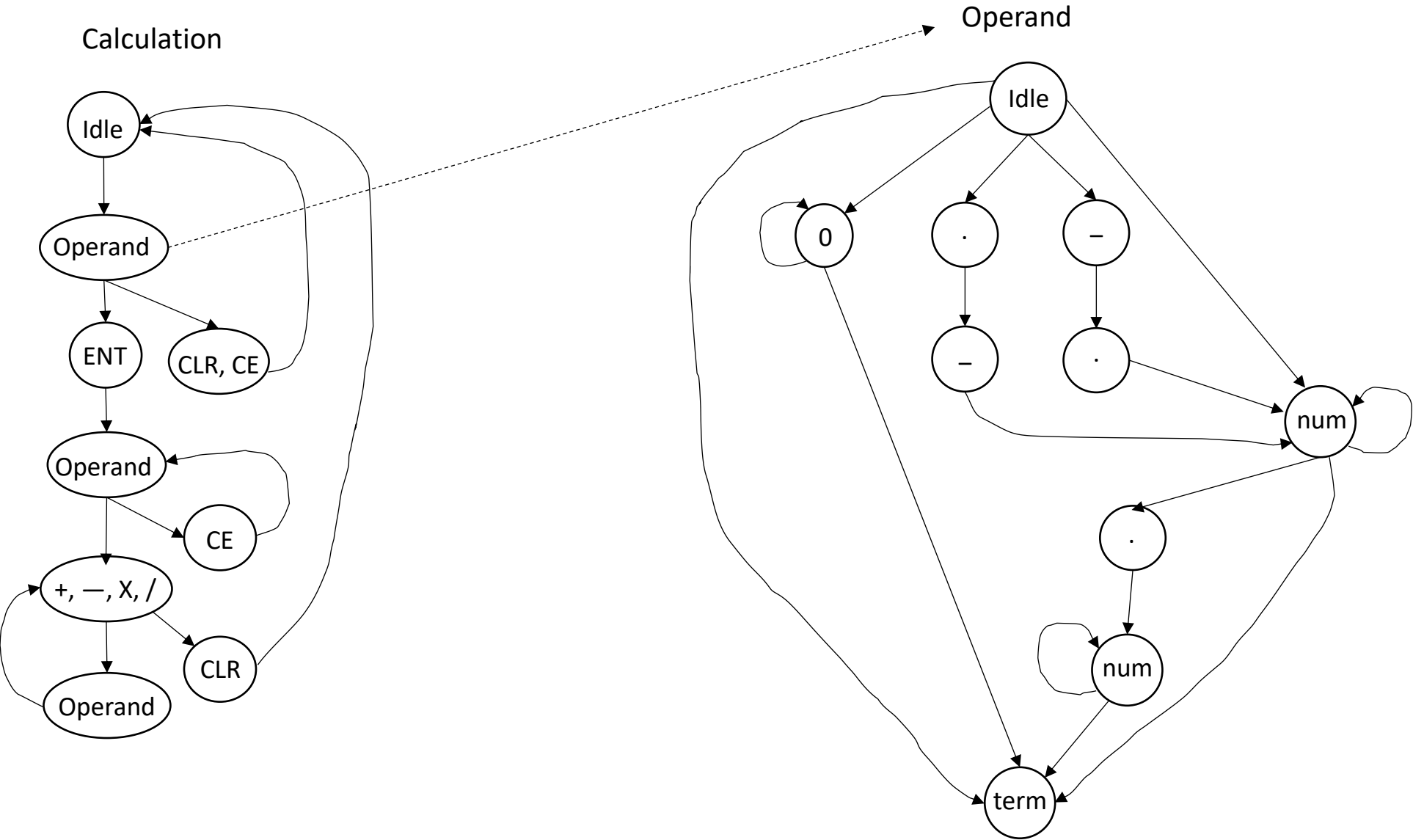


ArithExp

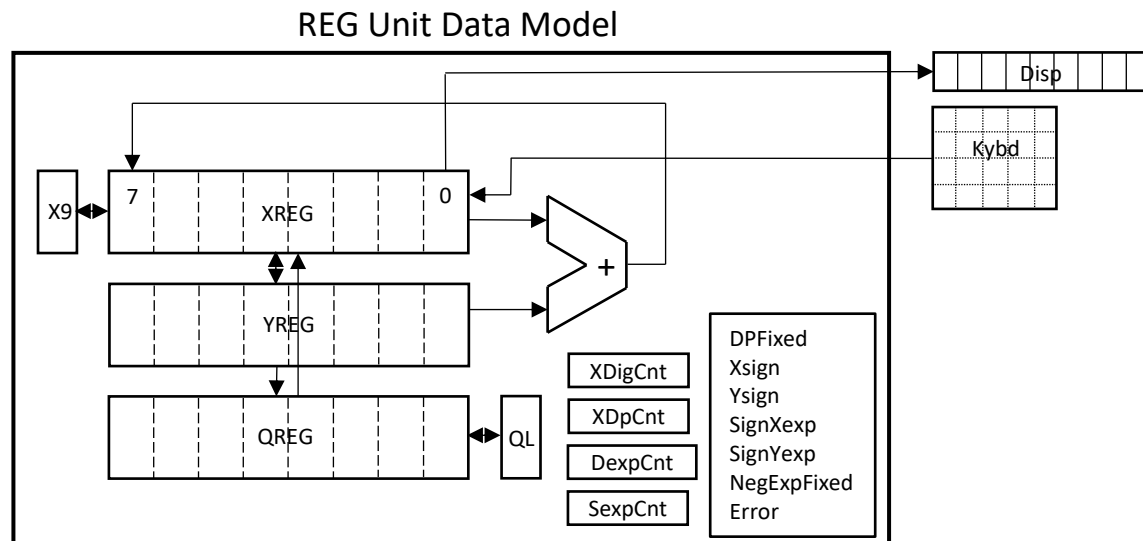
* Branch Condition



Key Sequence For Operations



How Operands Are Entered



- XDigCnt – Counts numeric digits entered into Xreg
- XDpCnt – Counts digits after decimal point in Xreg
- DexpCnt – Computes difference between exponents in Xreg and Yreg
- SexpCnt – Computes sim of exponents in Xreg and Yreg
- NegExpFixed – Indicates that accounting for negative exponent is done

Key	Yreg	Xreg/Disp								Xdig	XDp	Sexp	Dexp	DpFix	NEF	PhsSt
	76543210	7	6	5	4	3	2	1	0							
num									1.	1	0	1	1	0	0	op1ph
num									1 2.	2	0	2	2	0	0	op1ph
Dp									1 2.	2	0	2	2	1	0	op1ph
num									1 2.3	3	1	2	2	1	0	op1ph
num									1 2.3 4	4	2	2	2	1	0	op1ph
Ent	12340000	1	2	3	4	0	0	0	0	8	6	1	1	1	0	eop1ph
num	12340000								9.	1	0	2	0	0	0	op2ph
num	12340000								9 8.	2	0	3	-1	0	0	op2ph
num	12340000								9 8 7.	3	0	4	-2	0	0	op2ph
Dp	12340000								9 8 7.	3	0	4	-2	1	0	op2ph
num	12340000								9 8 7.6	4	1	4	-2	1	0	op2ph
+	12340000	9	8	7	6	0	0	0	0	8	5	3	-1	1	0	oprph
Dp									0.	1	0	0	0	1	0	op1ph
num									0.0	2	1	-1	-1	1	0	op1ph
num									0.0 0	3	2	-2	-2	1	0	op1ph
num									0.0 0 0	4	3	-3	-3	1	0	op1ph
num									0.0 0 0 1	5	4	-3	-3	1	1	op1ph
num									0.0 0 0 1 2	6	5	-3	-3	1	1	op1ph
num									0.0 0 0 1 2 3	7	6	-3	-3	1	1	op1ph
Ent	12300000	0	0	0	0	1	2	3	0	8	7	-4	-4	1	1	eop1ph
num	12300000								9.	1	0	-3	-5	0	0	op2ph
Dp	12300000								9.	1	0	-3	-5	1	0	op2ph
num	12300000								9.8	2	1	-3	-5	1	0	op2ph
num	12300000								9.8 7	3	2	-3	-5	1	0	op2ph
+	12300000	9	8	7	0	0	0	0	0	8	7	-4	-4	1	0	oprph

Dec	Bin
-14	0010
-13	0011
-12	0100
-11	0101
-10	0110
-9	0111
-8	1000
-7	1001
-6	1010
-5	1011
-4	1100
-3	1101
-2	1110
-1	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110

MicroCode Sample

7.5.3			I[5:0], I23 1[5:0], 3[7]		I[3:0] 1[3:0]	I[22] 3[6]	I[13:8] 2[5:0]	I[7:5] 1[7:5]	I[18:16] 3[2:0]	I[21:19] 3[5:3]	I[3:0] 1[3:0]	I[14] 2[6]	I[15] 2[7]	Logisim uROM Image D[7:0] I[15:8] [23:16]					
Addr	Hex	Label	SEQ		DSKY		REG					AddSub		1[7:0]	2[7:0]	3[7:0]	4		
			Bra	Target	Sel[3:0]	KeyAck	RegCmd	SelCtl1	SelCtl2	SelCtl3	KeyMCntSel	Sub_AddB	CarryEn	v2.0 raw			Required for Logisim		
675	2A3	ENTOP1	BRSignXexp	SETOP1DPFIR										3E	A9	02		If Op1 has neg exponent (DPFirst) then save in SignYexp (	
678	2A6		JMP	TSTDIGEMPTY										10	AC	02		Otherwise goto test for empty X reg	
681	2A9	SETOP1DPFIR							StatBitEn	StatBit1	SignYexp			05	00	1B			
684	2AC	TSTDIGEMPTY	BRDigEmpty	ENTERZERO										32	B5	02		If no digit hit yet, Jump to enter0	
687	2AF		BRDPFree	ADJEXPFIXDP										37	BE	02		If DP was never hit then goto adjust exp and fixDP	
690	2B2		JMP	LEFTJUST										10	C7	02		Otherwise go to left justify	
693	2B5	ENTERZERO							CntrEn	Cntrlncr	XDigCnt			01	00	36		Incr Xreg Dig count (enter a 0)	
696	2B8								StatBitEn	StatBit1	FixDP			00	00	1B		and fix the DP	
699	2BB		JMP	LEFTJUST										10	C7	02		then left justify	
702	2BE	ADJEXPFIXDP							CntrEn		XYSexpCnt			04	00	06		If X is whole integer, fix DP and adjust exponent	
705	2C1								CntrEn		XYDexpCnt			03	00	06			
708	2C4								StatBitEn	StatBit1	FixDP			00	00	1B			
711	2C7	LEFTJUST	BRDigFull	COPYXY										31	D6	02		Left justify X until Xdig is full	
714	2CA						LeftJustX	SelXData			0			00	02	00		Shift left with 0's in at msd	
717	2CD								CntrEn	Cntrlncr	XDigCnt			01	00	36		and adjust Xdp and Xdig count to match	
720	2D0								CntrEn	Cntrlncr	XDPCnt			02	00	36			
723	2D3		JMP	LEFTJUST										10	C7	02			
726	2D6	COPYXY				InitLp8								00	00	40		Copy Xreg to Yreg	
729	2D9		BRLoop8NZ	COPYXYE										3F	DC	02			
732	2DC	COPYXYE					CopyXY	SelXReg	SelXXlsl	SelYXlsl				20	05	28			
735	2DF		BRLoop8NZ	COPYXYE										3F	DC	02			
738	2E2		BRSignX	CPSIGXSIGY										34	E8	02		Copy Xsign to Ysign	
741	2E5		JMP	RMLDZEROS										10	EB	02			
744	2E8	CPSIGXSIGY							StatBitEn	StatBit1	SignY			04	00	1B			
747	2EB	RMLDZEROS				InitLp8								00	00	40		Loop to remove leading zeros from fractional args. Clea	
750	2EE	BRN1E	BRLoop8NZ	RLZENTERE										3F	F4	02		Burn loop 1 of 9 and jump to entry point	
753	2F1	RLZTOPE					RmLdZeroX	SelXData			0			00	02	00		Shift Xmsd out the top and zero in from the bottom	

MicroCode Fields, Mnemonics and Codes

	Jmp/Bra	1[4]	1[3:0]	RegCmd	2[5:0]	SelCtl1	1[7:5]	SelCtl2	3[2:0]	CtlSel3	3[5:3]	KeyMCntSel	1[3:0]
	JMP	1	0000	CircX	000001	SelXReg	001	ClrXPh1	001	SelQLQlSd	001	0	0000
BR0	BRDecEven	1	0000	DPAdjX	000001	LoadX9	010	ClrYPh2	010	SelYQYlSd	010	1	0001
BR1	BRDigFull	1	0001	EntNumX	000010	SelXQmsd	011	StatBitEn	011	StatBit1	011	2	0010
BR2	BRDigEmpty	1	0010	LeftJustX	000010	LoadQL	100	SelXYlSd	100	SelX9Xmsd	100	3	0011
BR3	BRDintOvrfl	1	0011	DPAdjY	000100	SHDexpXDP	101	SelXSum	100	SelXYlSd	101	4	0100
BR4	BRSignX	1	0100	CopyXY	000101	Reserved1	110	NxtPh	101	CntrlIncr	110	5	0101
BR5	BRSignY	1	0101	ExchXY	000101	Reserved2	111	CntrEn	110	RowHold	111	6	0110
BR6	BRInOp2Ph	1	0110	AddSub	000101	HWINTEN	000	SHSexpDexp	111	SelYZero	000	7	0111
BR7	BRDPFree	1	0111	DivXY10	000101	SelXData	000	SelXX9	000	SelQZero	000	8	1000
BR8	BRQLZero	1	1000	ShrX9X8QDiv10	010001	SelXSumX9	000	SelXXlSd	000	SelX9Zero	000	9	1001
BR9	BRX9Zero	1	1001	CopyYQ	010100	ExtraSelCtl1	000	ExtraSelCtl1	000	SelQLZero	000	BlankNum	1111
BR10	BRDpOvrfl	1	1010	CopyQX	100010	ExtraSelCtl2	000	ExtraSelCtl2	000	ExtraSelCtl3	000	XDigCnt	0001
BR11	BRNoKeyDet	1	1011	ApendDivdn	000010	ExtraSelCtl3	000					XDPCnt	0010
BR12	BRIsNeg	1	1100	RmLdZeroX	000010	ExtraSelCtl4	000	ALUCmd	2[7:6]	SEQ/DSKY	3[7:6]	XYDexpCnt	0011
BR13	BRCarryO	1	1101	CopyQ0QL	100000			SubXY	x1	HWINTEN	10	XYSexpCnt	0100
BR14	BRSignXexp	1	1110	ShrX	000001	BR*	001	CarryEn	1x	KeyAck	01	X9Cnt	0101
BR15	BRLoop8NZ	1	1111	ShlX	000010					InitLP8	01	QLCnt	0110
	BRSignYexp	1	0110	CopyYX	000101							FixDP	0000
	BrNegExpTerm	1	0101	ExtraCmd3	000000							SignXexp	0001
	ExtraBR2	0	0000									NegExpTerm	0010
	ExtraBR3	0	0000									SignX	0011
	HWINTEN	0	0000									SignY	0100

ImmData asserts on Ph2 of each cycle

SelCtl1, SelCtl2, RegCmd, ALUCmd assert on Ph4 and is valid only Ph4

SelCtl3, KeyAckB, HWIntEn Assert on Ph4 of each cycle

SelCtl3 Can assert simultaneous with JMP/BR*. SelCtl1/2 cannot.

MicroInstruction Fields

SelCtl1 (I5br:I7)

SelXReg	001	Select X reg MSD source from X reg or Y reg group
LoadX9	010	Load X9 source into X9 reg
SelXQmsd	011	Select X reg source from Q Dig7
LoadQL	100	Load QL source into QL reg
SHDexpXDP	101	Load Dexp counter value into XDP counter
Reserved1	110	Currently unassigned
Reserved2	111	Currently unassigned
SelXData	000	Select X LSD reg source from D[3:0]
SelXSumX9	000	Select X reg MSD source from Sum or X9 group

SelCtl2 (I16:I18)

SelQLQlsd	001	Select QL reg source from Q reg LSD
SelYQYlsd	010	Select Y and Q reg MSD sources from Y reg LSD
StatBit1	011	Set selected status bit to 1
SelX9Xmsd	100	Select X9 reg source from X reg MSD
SelYXlsd	101	Select Y reg source from X reg LSD
Cntrlncr	110	Increment selected and enabled counter
RowHold	111	DSKY row latch disable (hold)
SelYZero	000	Select Y reg MSD source from 0
SelQZero	000	Select Q reg MSD source from 0
SelX9Zero	000	Select X9 reg source from 0
SelQLZero	000	Select QL reg source from 0

SelCtl3 (I19:I21)

ClrXPh1	001	Clear X reg domain devices
ClrYPh2	010	Clear Y reg domain devices
StatBitEn	011	Enable status latches for write
SelXYlsd	100	Select X reg MSD source from Y reg LSD
SelXSum	100	Select X reg MSD source from SUM
NxtPh	101	Advance tracking state machine state
CntrlEn	110	Enable selected counter for up/down advance
SHSexpDexp	111	Load Sexp counter value into Dexp
SelXX9	000	Select X reg MSD source from X9 reg
SelXXlsd	000	Select X reg MSD source from X reg LSD

RegCmd (I8:I13)

CircX	000001	Shift X reg from MSD toward LSD
DPAdjX	000001	
ShrX	000001	
EntNumX	000010	Shift X reg from LSD toward MSD
LeftJustX	000010	
ApendDivdn	000010	
RmLdZeroX	000010	
DPAdjY	000100	Shift X reg from MSD toward LSD
CopyXY	000101	Shift X and Y reg from MSD toward LSD
ExchXY	000101	
AddSub	000101	
DivXY10	000101	
ShrX9X8QDiv10	010001	Shift X and Q reg from MSD toward LSD
CopyYQ	010100	Shift Y and Q reg from MSD toward LSD
CopyQ0QL	100000	Shift Q reg from LSD toward MSD
CopyQX	100010	Shift X and Q reg from LSD toward MSD

Branches (D3:D0)

JMP	NA	Unconditional Branch
BRDecEven	0000	Branch if DexpCnt=0
BRDigFull	0001	Branch of DigCnt=8
BRDigEmpty	0010	Branch if DigCnt=0
BRDintOvrfl	0011	Branch if Dexp>=8
BRSignX	0100	Branch if SignX status bit set
BRSignY	0101	Branch if SignY status bit set
BRInOp2Ph	0110	Branch if in or beyond Op2Ph tracking state
BRDPFree	0111	Branch of DP status bit has not been set
BRQLZero	1000	Branch if Qlreg=0
BRX9Zero	1001	Branch if X9reg=0
BRDpOvrfl	1010	Branch if XDpCnt>=8
BRNoKeyDet	1011	Branch if no key is currently pressed
BRIsNeg	1100	Branch if arithmetic (SUM) sign bit is set
BRCarryO	1101	Branch if arithmetic carry out bit is set
BRSignXexp	1110	Branch if Op2's exponent sign status bit is set
BRLoop8NZ	1111	Branch if LoopCnt>0
BRSignYexp	0110	Branch if Op1's exponent sign status bit is set
BrNegExpTerm	0101	Branch if current negative exponent calculated

Immediate Data Labels (D3:D0)

0	0000	Immediate data 0
1	0001	Immediate data 1
2	0010	Immediate data 2
3	0011	Immediate data 3
4	0100	Immediate data 4
5	0101	Immediate data 5
6	0110	Immediate data 6
7	0111	Immediate data 7
8	1000	Immediate data 8
9	1001	Immediate data 9
XDigCnt	0001	Select the XDigit counter
XDPCnt	0010	Select the XDP counter
XYDexpCnt	0011	Select the Dexp counter
XYSexpCnt	0100	Select the Sexp counter
X9Cnt	0101	Select the X9 counter
QLCnt	0110	Select the QL counter
FixDP	0000	Select the DP status bit
SignXexp	0001	Select the Op2 exponent sign status bit
NegExpTerm	0010	Select the negative exponent accumulation terminated status bit
SignX	0011	Select the Op2 sign status bit
SignY	0100	Select the Op1 sign status bit
SignYexp	0101	Select the Op1 exponent sign status bit
AssertError	0110	Select the Error status bit

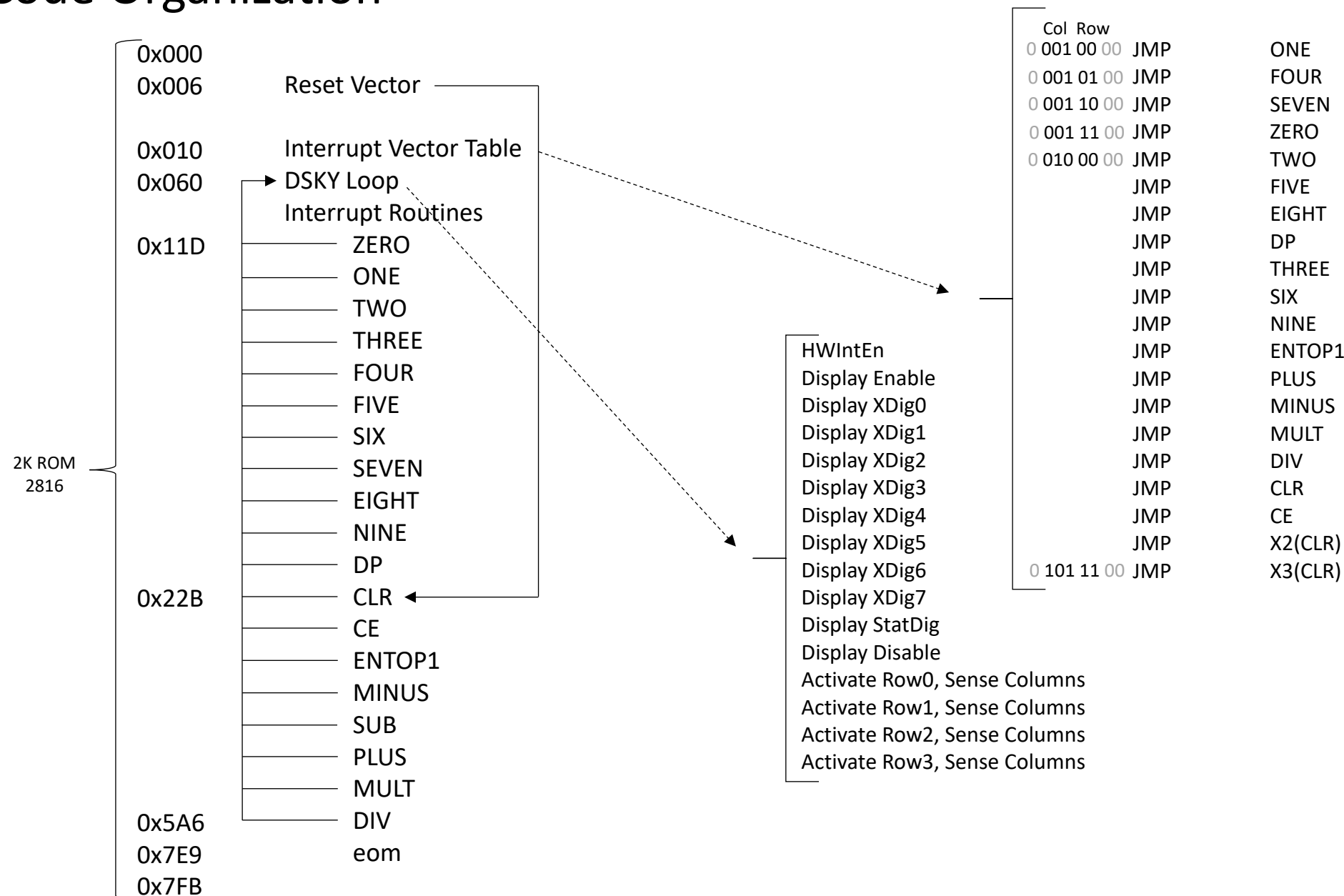
Add/Sub (I14:I15)

SubAddB	0	Add Digit 0 of Y register with Digit 0 of X register
	1	Subtract Digit 0 of X register from Digit 0 of Y register
CarryEn	0	Disable Carry In to Adder/Subtractor for one instruction
	1	Enable Carry In to Adder/Subtractor for one instruction

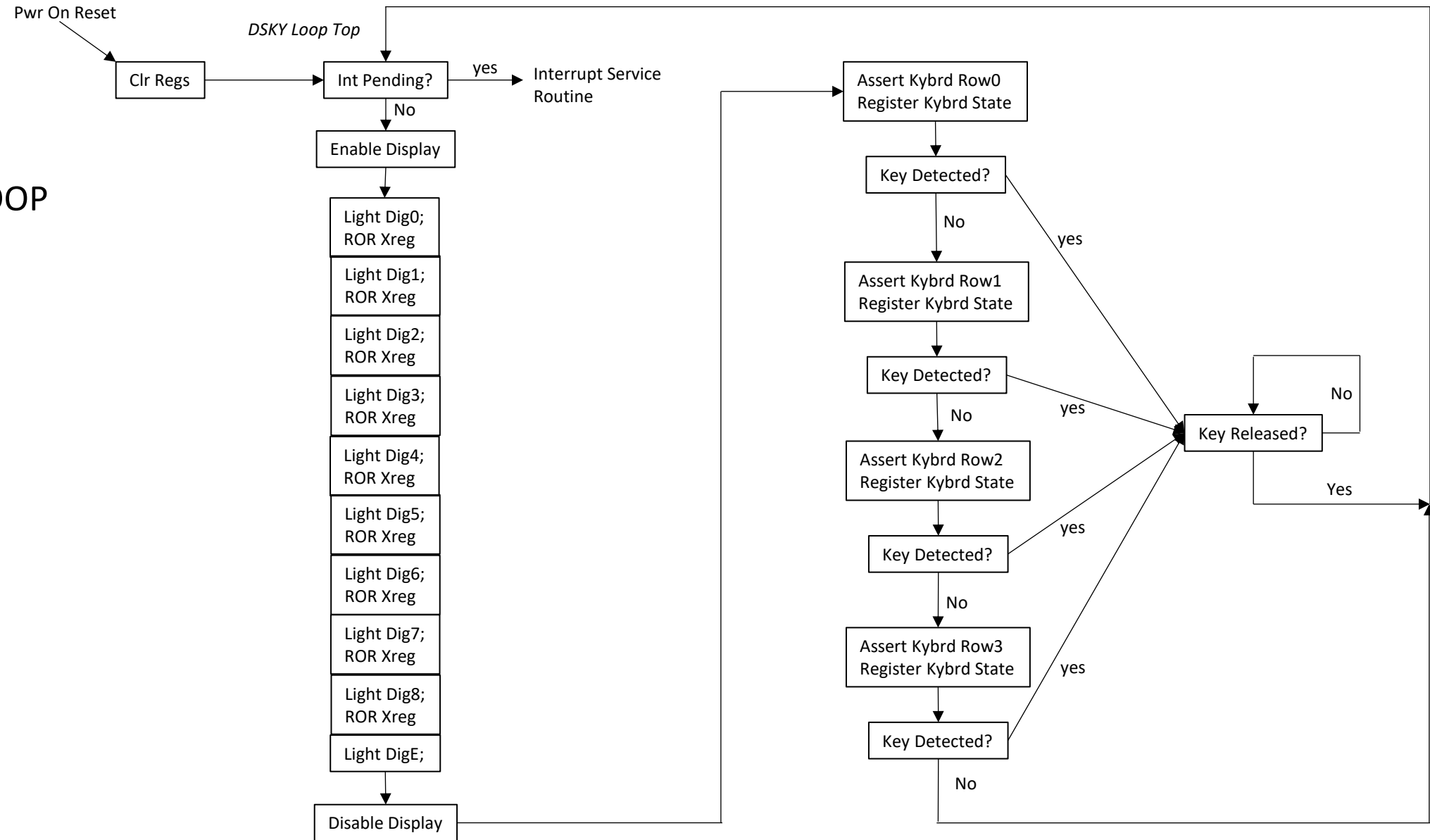
SEQ/DSKY (I22:I23)

KeyAck	0	NA
	1	Write state of keyboard into keyboard register, reset sequencer loop counter
HWIntEn	0	NA
	1	Enable branch to interrupt service routine if HWInt is active for one instruction

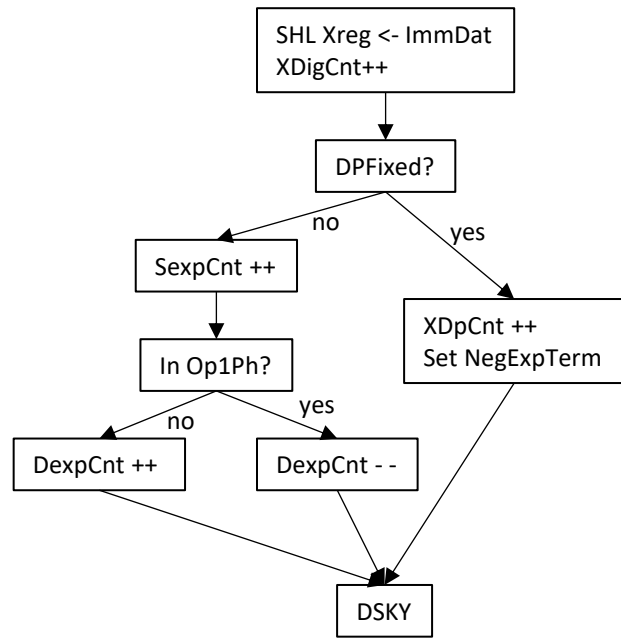
MicroCode Organization



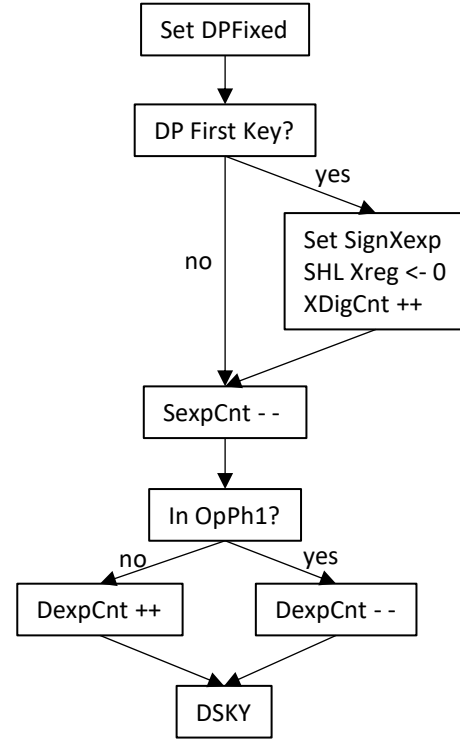
DSKY LOOP



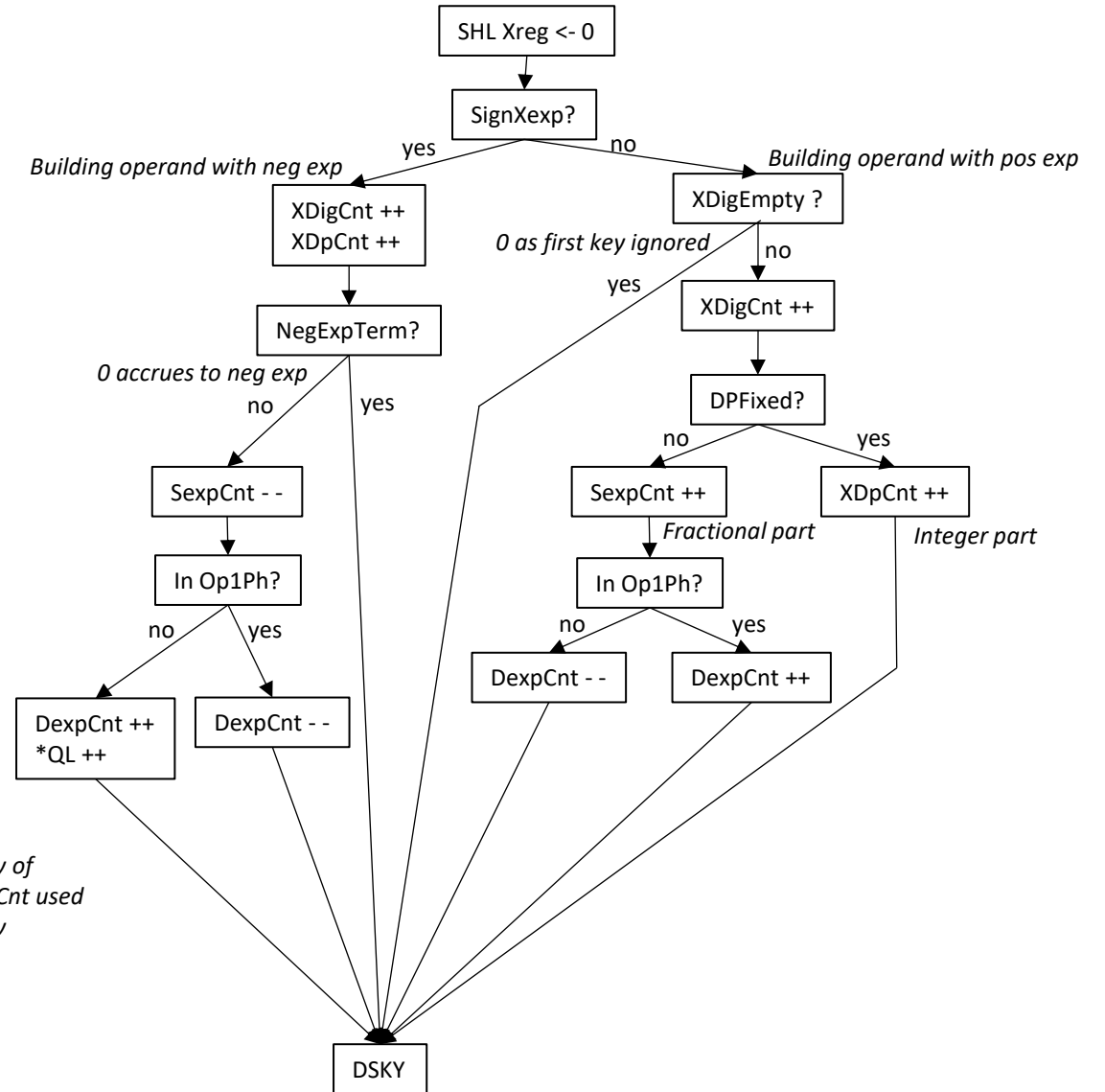
1-9 Service Routine



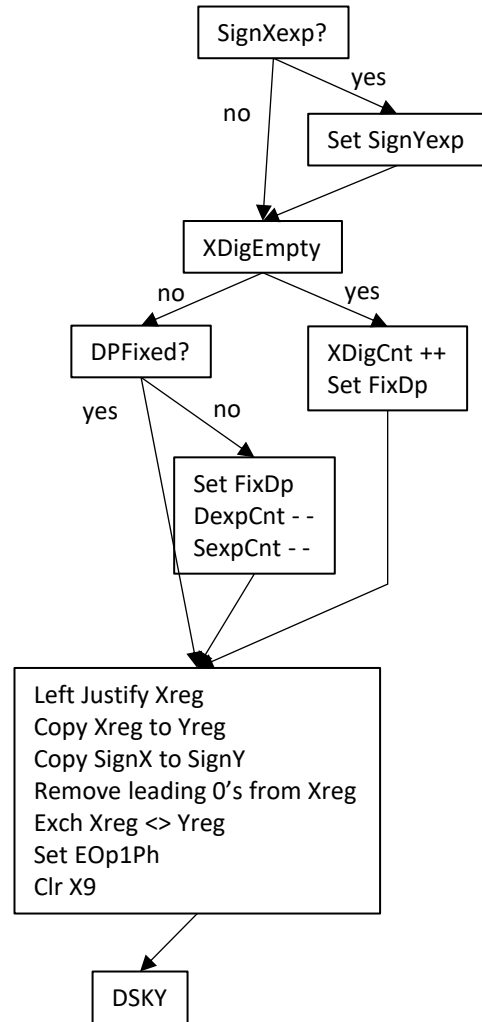
Dp Service Routine



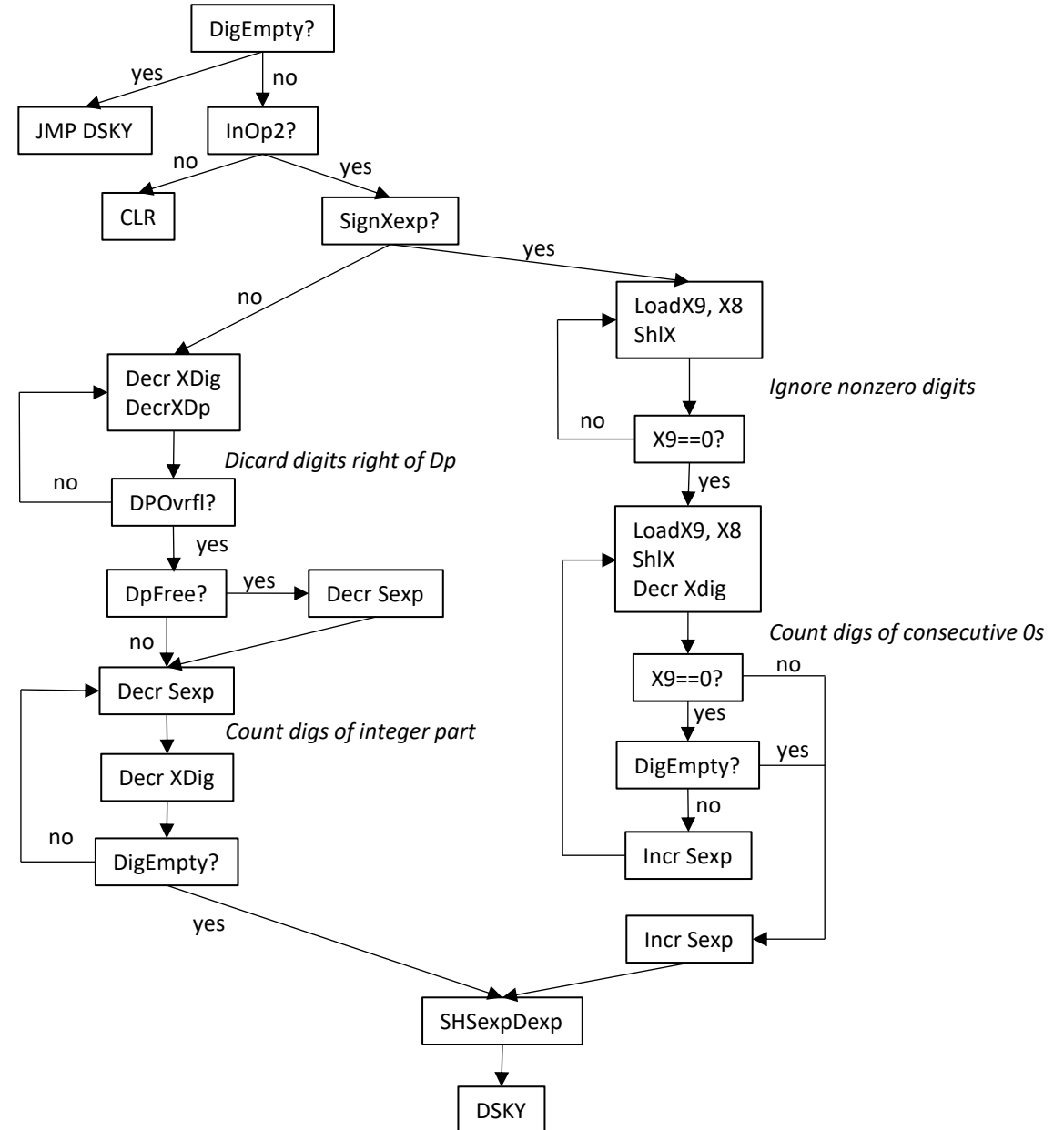
Zero Service Routine



Enter Op1 Service Routine



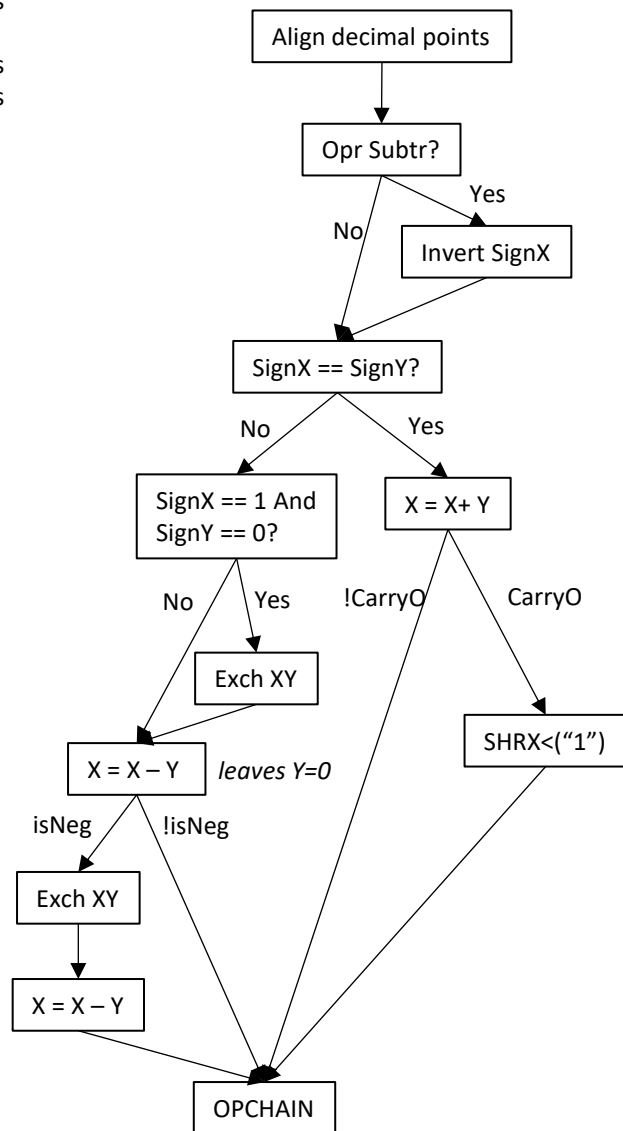
CE Service Routine



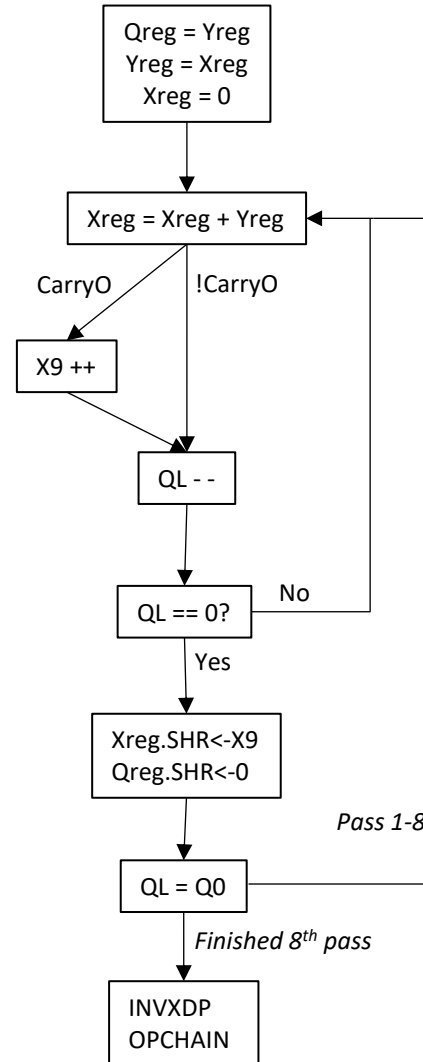
Basic Register Model

Xreg : Array[8:1] of BCD digits
 X9 : BCD digit
 Yreg : Array[8:1] of BCD digits
 Qreg : Array[8:1] of BCD digits
 QL : BCD digit
 Xdp : Integer[2:0]
 DpFixed : Boolean
 Sexp : Integer[3:0]
 Dexp : Integer[3:0]
 NegExpDpFixed : Boolean

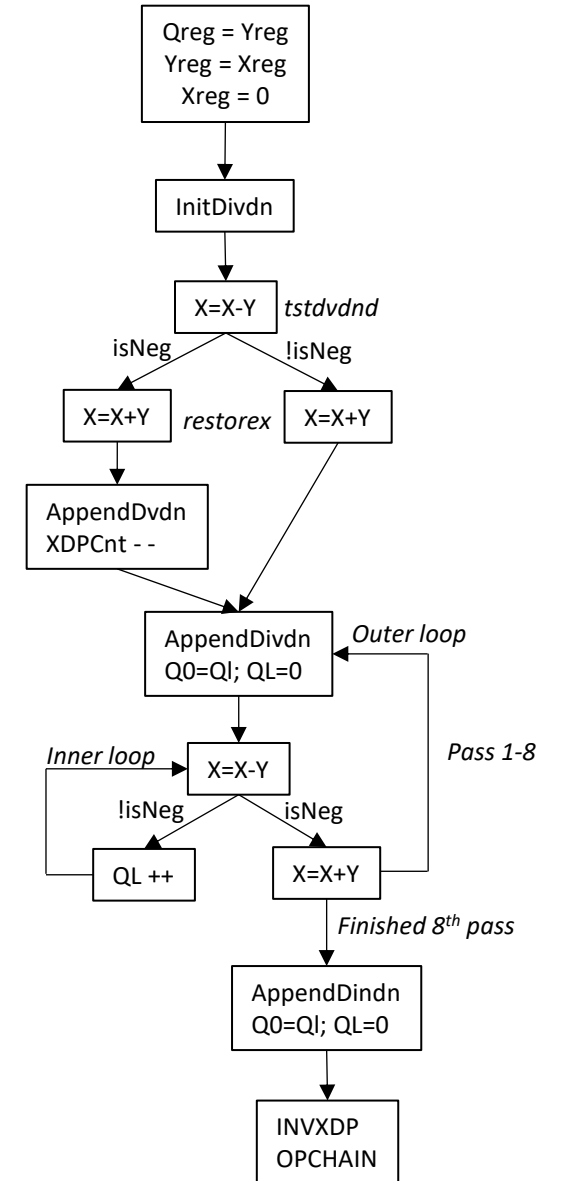
Add or Sub



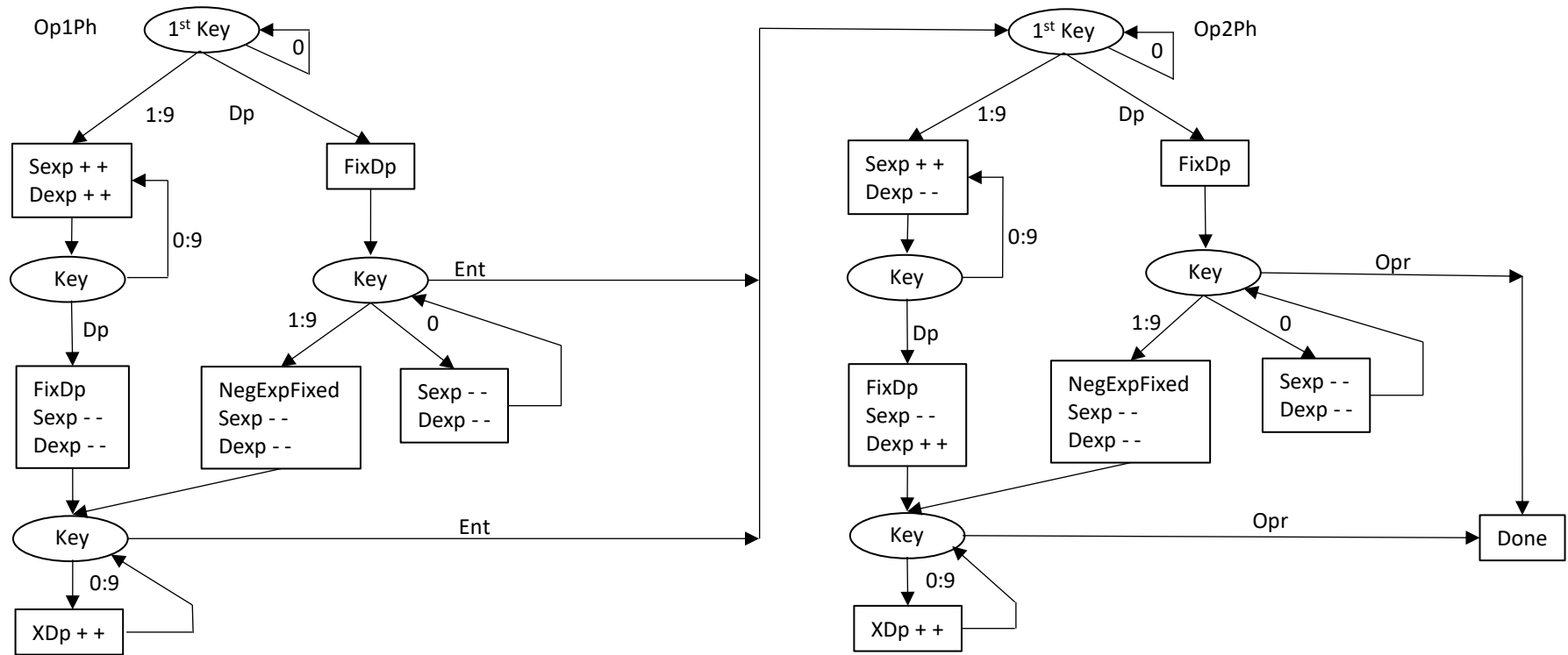
Mult



Div



Exponent Computation (XDp Placement)



Add/Sub:

If $Dexp \leq 0$, then $XDp = XDp(Op2)$ else $XDp = XDp(Op2) + Dexp$.

Mult

Pos Exp: XDp placement is initialized by Sexp. Incremented once (XDp shifted right one digit) if the final Mult loop leaves a non-zero accumulation of CarryOs in X9.

Neg Exp: XDp placed in left most position. Sexp number of leading zeros right shifted into MSD of product, losing right most digits for each shift.

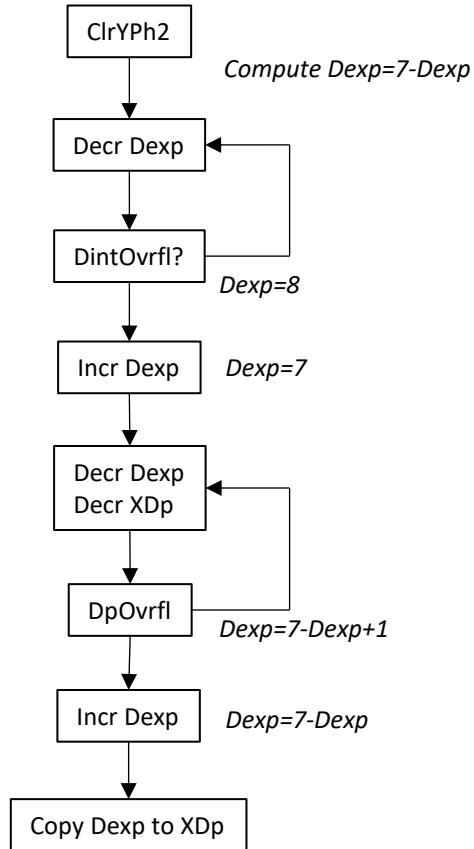
Div:

Pos Exp: XDp placement is initialized by Dexp. Decremented once (XDp shifted left one digit) if the minimum number of dividend digits needed to subtract the divisor with a positive result is one more than the divisor digits.

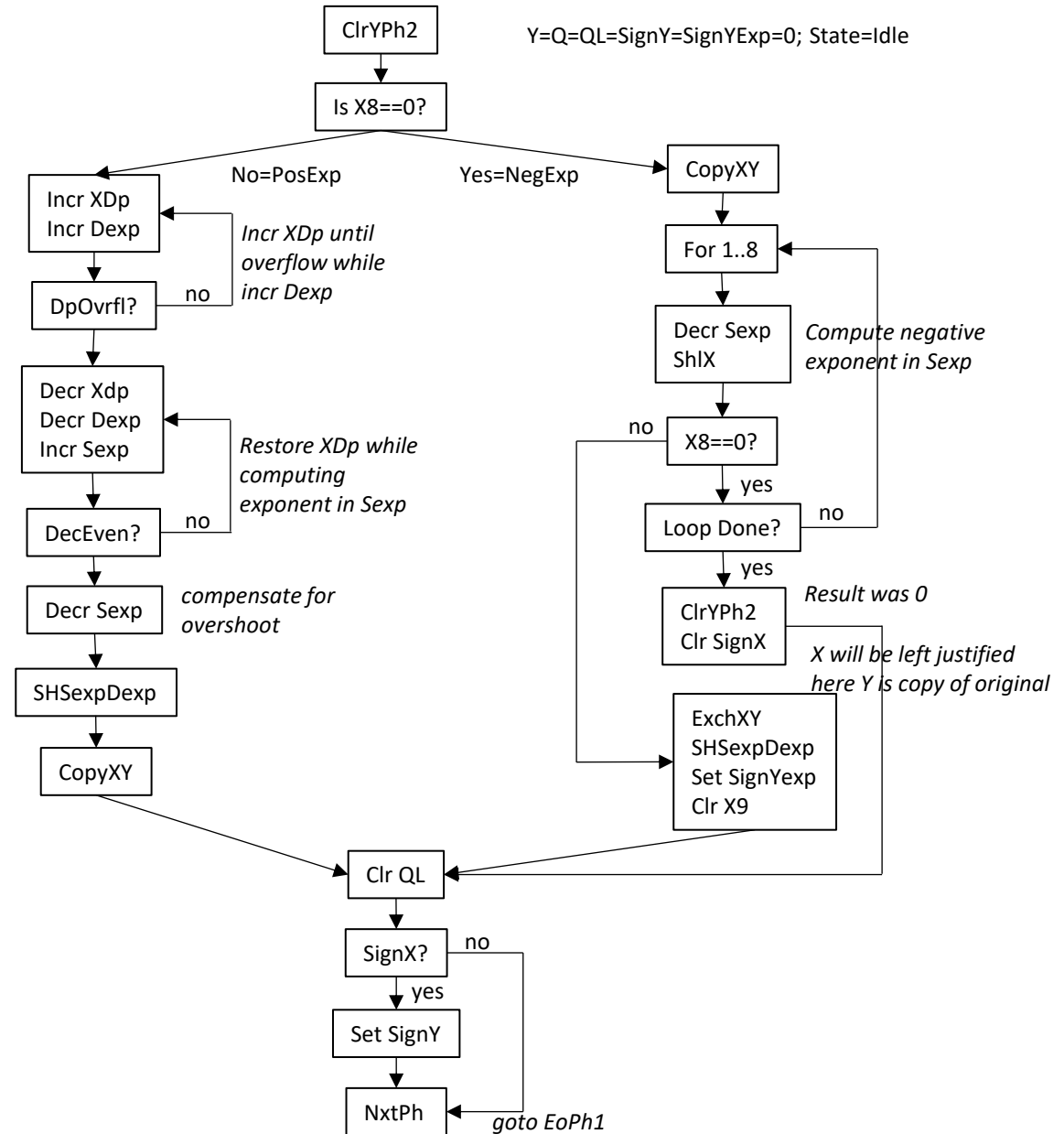
NegExp: XDp placed in left most position. Dexp number of leading zeros right shifted into MSD, losing rightmost digits for each shift.

Operation Chaining (Post processing the result of an operation to support chaining)

Invert XDP



Y=Q=QL=SignY=SignYExp=0; State=Idle



DSKY Loop

```
Do Forever
  If HWInt Then
    JMP IntTbl[IV]    // Execute key routine
  Enable Disp
  For (Digit 0 to 7) // Light each digit
    Select Digit
    ROR Xreg
  Sel StatDigit
  Disable Disp
  For (Row 0 to 3)    // Scan keyboard
    If DetectKey
      Wait Until KeyReleased
```

1-9 Service Routine

```
SHL <num> into Xreg
Incr XdigCnt
If Dp_Not_Fixed then
  Incr SexpCnt
  If Op1Ph then
    Incr DexpCnt
  Else
    Decr DexpCnt
Else // Dp Fixed. Key hits no longer
  // accumulate exponents
  Incr XDpCnt
  Set NegExpTerm // Indicate negative exponent
  //accumulation terminated
```

Zero Service Routine

```
SHL "0" int Xreg
If Not SignXexp then // If Xreg is > 1
  If XDigEmpty then // And Xreg is empty
    JMP DSKY // Then key hit can be ignored
  Else // Building operand > 1
    Incr XdigCnt
    If DpFixed Then // Building the fractional part
      Incr XDpCnt
      JMP DSKY
    Else // Building the integer part
      Incr SexpCnt
      If Op1Ph Then
        Incr DexpCnt
      Else
        Decr DexpCnt
      JMP DSKY
  Else // Building operand < 1
    Incr XdigCnt
    Incr XDpCnt
    If NegExpTerm Then // If no more leading 0's then done
      JMP DSKY
    Else // Account for the larger negative exponent
      Decr SexpCnt
    If Op1Ph Then
      Decr DexpCnt
    Else
      Incr DexpCnt
  Incr QL // QL used to mirror DexpCnt count
  // of leading 0's (used by Div)
JMP DSKY
```

Dp Service Routine

```
Set FixDp
If Dp is first key hit then
  Set SignXexp
  SHL "0" into Xreg
  Incr XDigCnt
Decr SexpCnt
If Op1Ph then
  Decr DexpCnt
Else
  Incr DexpCnt
JMP DSKY
```

Enter Op1 Service Routine

```
If Dp was first key hit then // Xreg < than 1
  Set SignYexp bit
If no numeric key was hit then
  Incr XDigCnt; Set FixDp
Else
  If Dp was not hit then // Xreg is pure integer
    Ser FixDp; Dexp - -; Sexp - -
  While not XDigFull // Left justify Xreg
    SHL Xreg <- 0; XdigCnt ++; XDPCnt ++
  Copy_Xreg_into_Yreg
  Copy SignX into SignY
  X9 <- Xmsd // Remove leading zeros. Does nothing
  //unless operand is < 1.
  While X9 == 0 // The operand in X is mantissa
    X9 <- SHL Xreg <- 0 // The operand in Y is displayable version
  Exchange Xreg<>Yreg
  NxtPh // Move to EOp1Ph
  Clear X9
  JMP DSKY
```

MINUS

```
If Not Op2Ph Then // If not in Op2 then "-" means sign bit
    Set SignX
    JMP DSKY
Else // If in Op2 then "-" means subtract
    Invert_SignX
PLUS
If DigEmpty Then // If no key hit for Op2 then enter "0"
    Incr XDigCnt
    Set FixDp
Else
    If DpFree Then // Op2 is pure integer, fix Dp and adj exp
        Set FixDp
        Decr SexpCnt
        Incr DexpCnt
    Left_Justify_Xreg
    While X8 == 0 // Remove leading 0's from Op2<1
        SHL Xreg, 0
        Incr XDpCnt
    While DexpCnt != 0 // Get operands exponent-aligned (DpAdj)
        If DintOvrfl Then // Shift Yreg right, count Dexp up
            SHR Yreg, 0
            Incr DexpCnt
        Else // Shift Xreg right, count Dexp down
            SHR Xreg, 0
            Decr DexpCnt
    While DpOvrfl // Restore leading 0's to Op2, if any
        SHR Xreg; SHR Yreg
        Decr XDpCnt
    If (SignX XOR SignY) == 0 Then // If Op signs same, then ADD
        Xreg = Xreg + Yreg
        If CarryO Then // If final carry out then shift in X9, adj XDp
            Incr X9
            SHR Xreg, X9
            Decr XDpCnt
    Else // Op signs are different
        If (Sign X == 1 AND SignY == 0) Then // Yreg must be the negative op for ALU
            Exch Xreg<>Yreg
            Exch SignX<>SignY
        Xreg = Xreg - Yreg
    If DpOvrfl Then
        Set Error
    JMP DSKY
```

Add and Subtract Service Routine

OPADD (Xreg + Yreg)

```
SHR Xreg, Sum; SHR Yreg, 0; ADD // Shift Xreg and Yreg right,
SHR Xreg, Sum; SHR Yreg, 0; ADD, CarryEn // with Lsd's going through ALU
SHR Xreg, Sum; SHR Yreg, 0; ADD, CarryEn // and the Sum shifting into
SHR Xreg, Sum; SHR Yreg, 0; ADD, CarryEn // Xreg msd, and 0's shifting
SHR Xreg, Sum; SHR Yreg, 0; ADD, CarryEn // into Yreg msd
SHR Xreg, Sum; SHR Yreg, 0; ADD, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; ADD, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; ADD, CarryEn
```

OPSUB (Xreg - Yreg)

```
SHR Xreg, Sum; SHR Yreg, 0; SUB
SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
If IsNeg Then // Get 10's complement and BCD adjust
    Exch Xreg<>Yreg
    SHR Xreg, Sum; SHR Yreg, 0; SUB
    SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
    SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
    SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
    SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
    SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
    SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
    SHR Xreg, Sum; SHR Yreg, 0; SUB, CarryEn
    Set SignX // Since result is negative
    While X8 == 0 AND NOT DpOvrfl // Remove excess leading 0's, if any
        SHL Xreg, 0
        Incr XDpCnt
```

Mult Service Routine

```

If DigEmpty Then    // If no key hit for Op2 then enter "0"
    Incr XDigCnt
    Set FixDp
Else
    If DPFree Then // Else if Dp not hit then FixDp and adj exps
        Set FixDp
        Decr SexpCnt
        Incr DexpCnt
    Left_Justify_Xreg
    While X8 == 0    // Remove any leading 0's in Op2
        SHL Xreg, 0; Incr XDpCnt
    NxtPh            // Move to OprPh
    If (SignX XOR SignY) == 1 Then
        Set SignX
    Else
        Clear SignX
    Copy Yreg to Qreg // Get regs set up for Mult loop
    Copy Xreg to Yreg, clearing Xreg
    Mult_Loop -----
    If (SignXexp == SignYexp == 0 AND DpOvrfl) Then // Overflow handler
        Set Error
        JMP DSKY
    If (SignXexp == SignYexp == 1 AND DpOvrfl) Then // Underflow handler
        ClrXPh
        JMP DSKY
    If DpOvrfl Then // Product is <1
        While DpOvrfl // Insert leading 0's to product <1
            SHR Xreg, 0 // leaving XDP at 0 (7 after inv)
            Incr XDpCnt
        Else // Product is >1
            While X8 == 0 // Remove leading 0's to product >1, if any
                SHL Xreg, 0
                Decr XDpCnt // Move Dp to the left
    Invert XDpCnt
    JMP DSKY

```

```

Copy Q0 into QL; SHR Qreg
NxtPh // Move to MultDivPh
Shift SexpCnt into XDpCnt // SexpCnt has the DP placement for product
For 8 Loops
    While QL != 0
        SHR Xreg, Sum; SHR Yreg, Y0; ADD // Shift Xreg and Yreg right,
        SHR Xreg, Sum; SHR Yreg, Y0; ADD, CarryEn // with Xmsd getting getting the Sum
        SHR Xreg, Sum; SHR Yreg, Y0; ADD, CarryEn // and Ymsd circulating Ylsd
        SHR Xreg, Sum; SHR Yreg, Y0; ADD, CarryEn
        SHR Xreg, Sum; SHR Yreg, Y0; ADD, CarryEn
        SHR Xreg, Sum; SHR Yreg, Y0; ADD, CarryEn
        SHR Xreg, Sum; SHR Yreg, Y0; ADD, CarryEn
        If CarryO Then
            Incr X9
        Decr QL
    SHR Xreg, X9; SHR Qreg, 0
    Clr X9
    If X9 != 0 Then
        SHR Xreg, X9
        Incr XDpCnt

```

Div Service Routine

```
For 8 Loops          // Check for Op2==0
  Load X8 into X9
  If X9 == 0 Then
    SHR Xreg, X1
  Else Continue DPCHK
Set Error
JMP DSKY
DPCHK If NOT DpFixed Then // If Dp never entered
  Set FixDp          // Fix Dp and exp adj
  Decr SexpCnt
  Incr DexpCnt
  NxtPh              // Move to OprPh
If (SignX XOR SignY) == 0 Then // Compute sign of quotient
  Clear SignX
Else
  Set SignX
If Op2 < 1 Then      // Divide both ops by 10 if Xreg is full
  If DigFull Then
    SHR Xreg, 0; SHR Y, 0
Copy Yreg to Qreg
Copy Xreg to Yreg, clearing Xreg
Move to MultDivPh
Copy DexpCnt to XDpCnt // Exponent for quotient is DexpCnt
If SignXexp Then      // Remove the extra "0" in front of Dp
  Decr XDigCnt
  While QL != 0
    Decr QL          // QL is a copy of DexpCnt to use for removing leading 0's
    Decr XDigCnt
While NOT DigEmpty // Init dividend with XDigCnt digits of Qreg
  SHL Xreg, Q8; SHL Qreg, 0
  Decr XDigCnt
X = X-Y              // Begin test if one more dividend digit is needed
If IsNeg Then
  X = X+Y // Restore dividend
  SHL Xreg, Q8; SHL Qreg, 0 // Add one more digit to dividend
  Decr XDpCnt
Else
  X = X+Y // Restore dividend
```

```
For 8 Loops // outer loop
  SHL Xreg, Q8 // Append dividend with additional digit
  SHL Qreg, 0; Copy QL to Q0 // Accumulate quotient
  Clr QL
  While 1 // Inner loop
    X = X - Y
    If isNeg Break
    Incr QL
    X = X+Y
  SHL Qreg, 0; Copy QL to Q0
  Clr QL
  Copy Qreg to Xreg // Copy quotient to Xreg
  If (SignYexp == 1 AND SignXexp == 0) Then // Underflow handler
    If DPOvrfl Then
      Clr Xreg
      JMP DSKY
  If (SignXexp == 1 AND SignYexp) == 0 Then // Overflow Handler
    If DPOvrfl Then
      Set Error
      JMP DSKY
  If NOT DPOvrfl Then // Quotient >=1; Done
    JMP DSKY
Repeat
  SHR Xreg, 0
  Incr XDpCnt
Until DPOvrfl
Invert XDpCnt
JMP DSKY
```

CE Service Routine

```
If Xreg is empty then
  JMP DSKY
If in Op1Ph then
  JMP CLR
Else
  If SignXexp then    // Back out the negative exponent
    While Xmsd != 0  // Shift out leading Fs
      SHL Xreg <- 0
    While (Xmsd == 0) AND (not XDigEmpty) // Count
      XDigCnt - -    // leading 0 digits and
      XYSexpCnt ++   // back out Op2 exponent
    XYSexpCnt ++
  Else                // Back out the positive exponent
    Repeat            // Remove fractional part of operand from XDigCnt
      XDpCnt - -
      XDigCnt - -
    Until (DpOvrfl)
    If not DpFixed then
      XYSexpCnt - -   // Back out exp correction integer part
    While (not DigEmpty) // Back out Op2 exponent
      XYSexpCnt - -
      XDigCnt - -
  Copy Sexp to Dexp
  ClrXPh1
  Initialize Xreg with "FFFFFFF0"
  JMP DSKY
```

CLR Service Routine

```
JMP RESET
```

Invert XDp

```
ClrYPh2
While not DintOvrfl
  XYDexpCnt ++
XYDexpCnt - -
While not DpOvrfl
  XYDexpCnt - -
  XYSexpCnt - -
XYDexpCnt ++
Copy Dexp to XDp
```

Operation Chaining

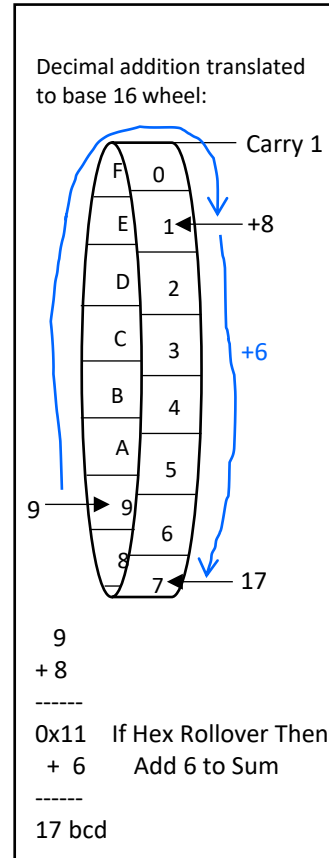
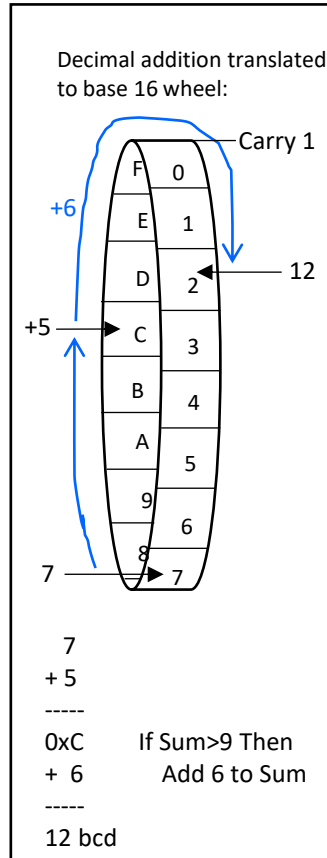
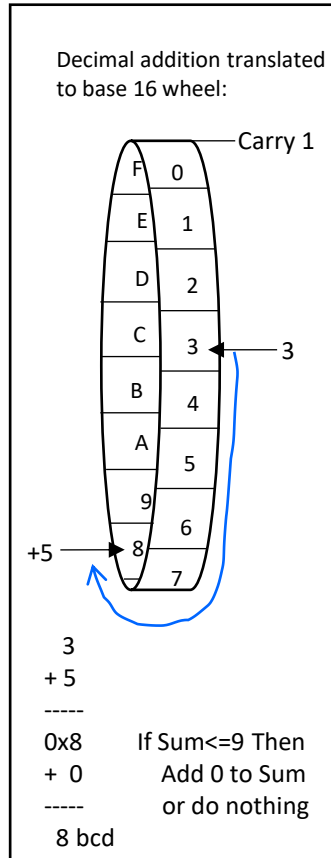
```
ClrYPh2; Clr QL // Dexp and all other y regs are 0
X9 <- Xmsd
If X9 == 0 then // Negative exponent
  Copy Xreg to Yreg
  For 1 to 8 // Compute negative exponent
    XYSexpCnt - - // decrement Sexp
    SHL Xreg // and shift left X
  X9 <- Xmsd // for every leading 0
  If X9 != 0 then // Done, X is left just mantissa
    Exch Xreg <> Yreg // X is Leftjust mantiss, Y is display copy
    Copy Sexp to Dexp
    Set SignYexp
    Clr X9
    Break to FINISH
  ClrYPh2 // Result was 0
  Clr SignX
Else // Positive exponent
  While (not XDpOvrfl) // compute Dexp=7-XDp
    XYDexpCnt ++
    XDPCnt ++
  While (not DecEven) // Restore XDp and compute exp in Sexp
    XDpCnt - - // It will overshoot by 1
    XYSexpCnt ++
    XYDexpCnt - -
  XYSexp ++ // Fix overshoot
  Copy Sexp to Dexp
  Exch Xreg <> Yreg
FINISH Clr X9
If SignX then
  Set SignY
Set EOPh1
JMP DSKY
```

BCD Addition

Given a 4-bit adder:

$$A_{10} + B_{10} + C_{in} \rightarrow (A_{BCD} + B_{BCD} + C_{in}) \bmod 16 = S_{16},$$

$$[(A_{BCD} + B_{BCD} + C_{in}) \bmod 16 \geq 16] = Co$$



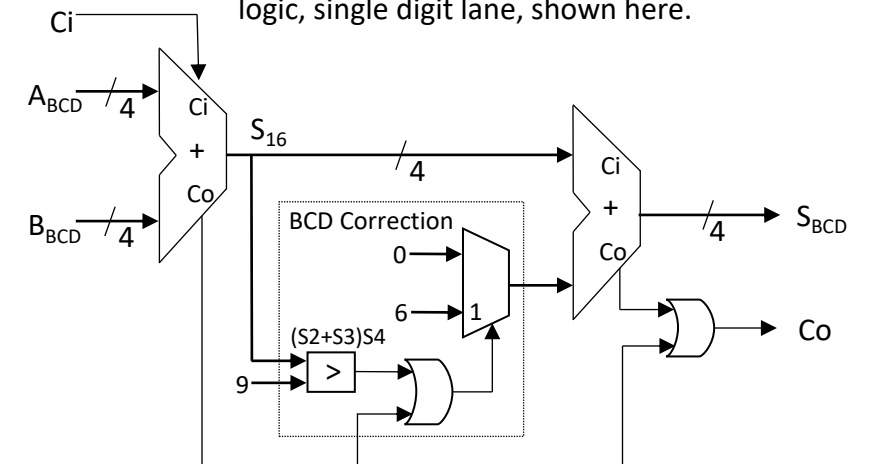
$(Op1 + Op2 + Ci)_{10}$	Sum_{16}	Co	Sum_{10}	Co
0	0	0	0	0
1	1	0	1	0
2	2	0	2	0
3	3	0	3	0
4	4	0	4	0
5	5	0	5	0
6	6	0	6	0
7	7	0	7	0
8	8	0	8	0
9	9	0	9	0
10	A	0	0	1
11	B	0	1	1
12	C	0	2	1
13	D	0	3	1
14	E	0	4	1
15	F	0	5	1
16	0	1	6	1
17	1	1	7	1
18	2	1	8	1
19	3	1	9	1

Sum of 2, 4-digit numbers

	Co	Ci	Co	Ci	Co	Ci	Co	
7693		0111	1		0110	1		1001
+9684		+1001			+0110			+1000
17377	1	0001			1101	1		0001
		+0110			+0110			+0110
								+0000
	1	0111	1	0011		0111		0111

Note: Carry out of either the main summer or the BCD corrector is a BCD carry out.

All of which leads to the idea of the BCD adder logic, single digit lane, shown here.



Number Complements

Radix complements and their sidekicks, diminished radix complements are a numeric tool allowing subtraction be performed by addition.

- A number N , in radix R has complement numbers.
- In particular a number N in radix R has its R 's complement and its diminished R 's complement, or $(R-1)$'s complement.
- The R 's complement is the $(R-1)$'s complement plus 1.
- The $(R-1)$'s complement is computed by taking each digit of N and subtracting it from $R-1$.

EX: If $R=2$ and $N=1110$, the 1's complement is 0001,
and the 2's complement is 0001 plus 1 or 0010.

9's and 10's Complement Subtraction

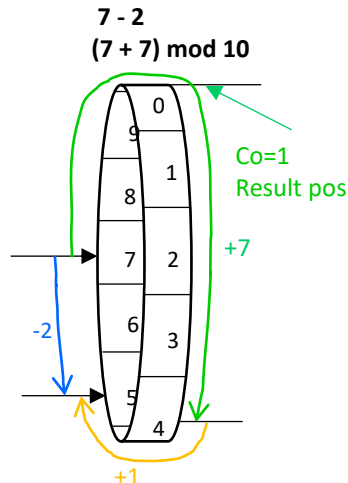
To take the 9's complement of a number, say 752, subtract each digit of the number from 9.

999
-752

247

To subtract two numbers using 9's complement arithmetic:

- Add the 9's complement of the subtrahend to the minuend.
- If there is a carry out of the most significant digit, then add 1 to the result. The result is positive.
- If there is no carry out of the most significant digit, the result is negative and must be 9's complemented.



$$5841 - 2963 = 2878$$

$$\begin{array}{r} 9999 \\ - 2963 \\ \hline 7036 \\ + 5841 \\ \hline 1 \mid 2877 \\ 1 \\ \hline 2878 \end{array}$$

9's complement of subtrahend

Add the minuend

Add MSD carry to sum

Difference is positive

$$4691 - 6849 = -2158$$

$$\begin{array}{r} 9999 \\ - 6849 \\ \hline 3150 \\ + 4691 \\ \hline 0 \mid 7841 \\ 9999 \\ - 7841 \\ \hline - 2158 \end{array}$$

9's complement of subtrahend

Add the minuend

Difference is negative

9's complement of difference

To take the 10's complement of a number, say 750, subtract each digit of the number from 9 and add 1 to the result.

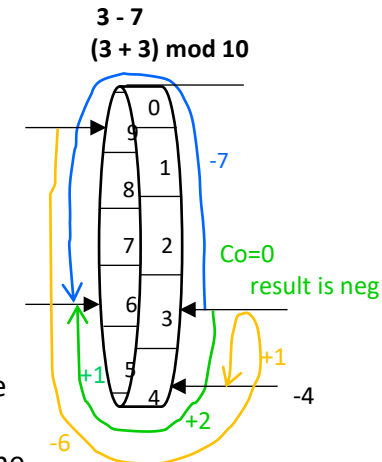
999
-750

249
+ 1

250

To subtract two numbers using 10's complement arithmetic:

- Add the 10's complement of the subtrahend to the minuend.
- If there is a carry out of the most significant digit, the result is positive.
- If there is no carry out of the most significant digit, the result is negative and must be 10's complemented.



$$5841 - 2963 = 2878$$

$$\begin{array}{r} 9999 \\ - 2963 \\ \hline 7036 \\ + 1 \\ \hline 7037 \\ + 5841 \\ \hline 1 \mid 2878 \end{array}$$

10's complement of subtrahend

Add the minuend

Difference is positive

$$4691 - 6849 = -2158$$

$$\begin{array}{r} 9999 \\ - 6849 \\ \hline 3150 \\ + 1 \\ \hline 3151 \\ + 4691 \\ \hline 0 \mid 7842 \\ 9999 \\ - 7842 \\ \hline 2157 \\ + 1 \\ \hline - 2157 \end{array}$$

10's complement of subtrahend

Difference is negative

10's comp of diff

10's Complement Arithmetic With A 4-Bit Binary Adder

For a 4-bit binary number, or hex digit H, the F's complement of H is $0xF - H$.

*Note that the F's complement of a 4-bit number is also the 1's complement of that 4-bit number.

For a BCD digit N, the 9's complement of N is the F's complement of N minus the number of hex digits not part of the decimal digit set (A,B,C,D,E,F), which is:

$0xF - N - 0x6$,

Or more specifically,
 $(0xF - N - 0x6) \bmod 16$.

And that is the same as:

$(0xF - N + 0xA) \bmod 16$

And

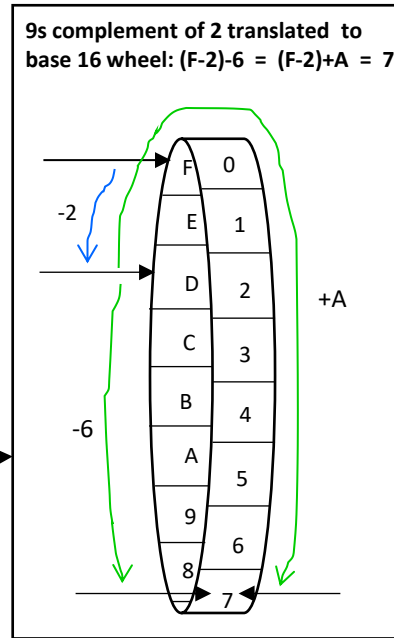
$(*1'scomp(N) + 0xA) \bmod 16$

Which is

$\sim N + 0xA \leftarrow$ 9's complement of N

The 10's complement of a number N is still just the 9's complement plus 1.

This suggests the idea of a BCD subtractor looks in part like the diagram on the right.

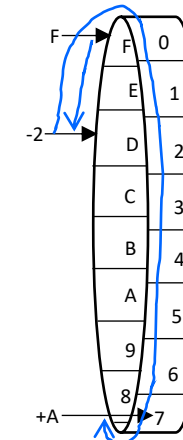


10's complement of 290

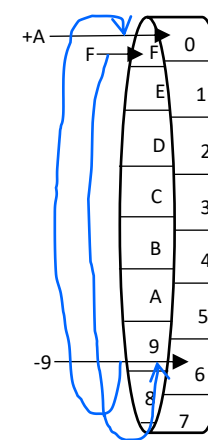
$$9-2 = (F-2+10) \bmod 16 = (\sim 2+A) \bmod 16$$

$$F-9+10 = (\sim 9+A) \bmod 16$$

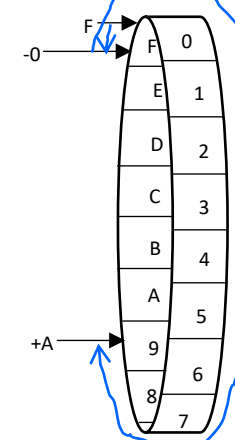
$$F-0+10 = (\sim 0+A) \bmod 16$$



7



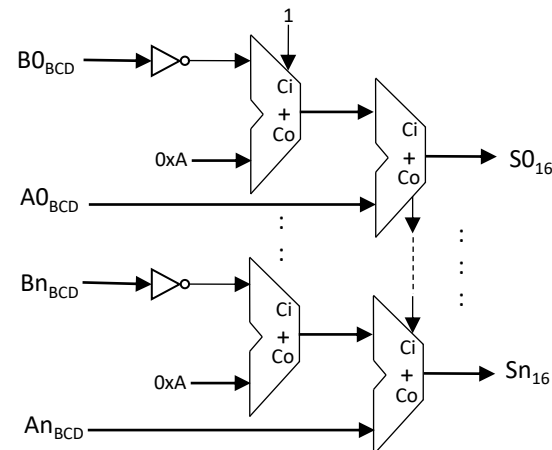
0



9

709
+ 1

710 = 10's complement of 290



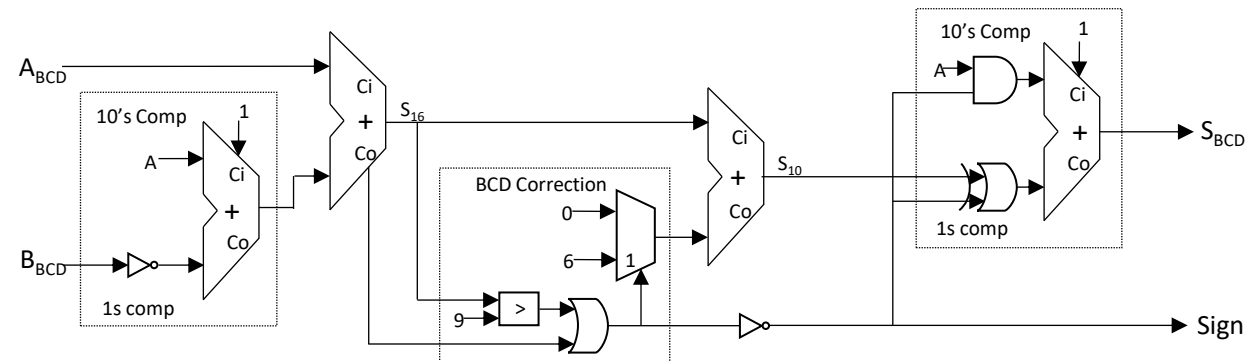
BCD Subtraction

		Co		Ci	Co		Ci			
A	47		0100			0111		minuend		
B	-28		-0010			-1000		subtrahend		
	19									
			1101			0111		1's complement of subtrahend		
			+1010			+1010	1	add 10 (and 1 more for LSD)		
		1	0111		1	0010		10's complement of subtrahend		
			+0100			+0111		add minuend		
			1011			1001		difference		
			+0110			+0000		bcd correction		
		1	0001			1001				
		Final carry=1 difference is pos (A>=B)								

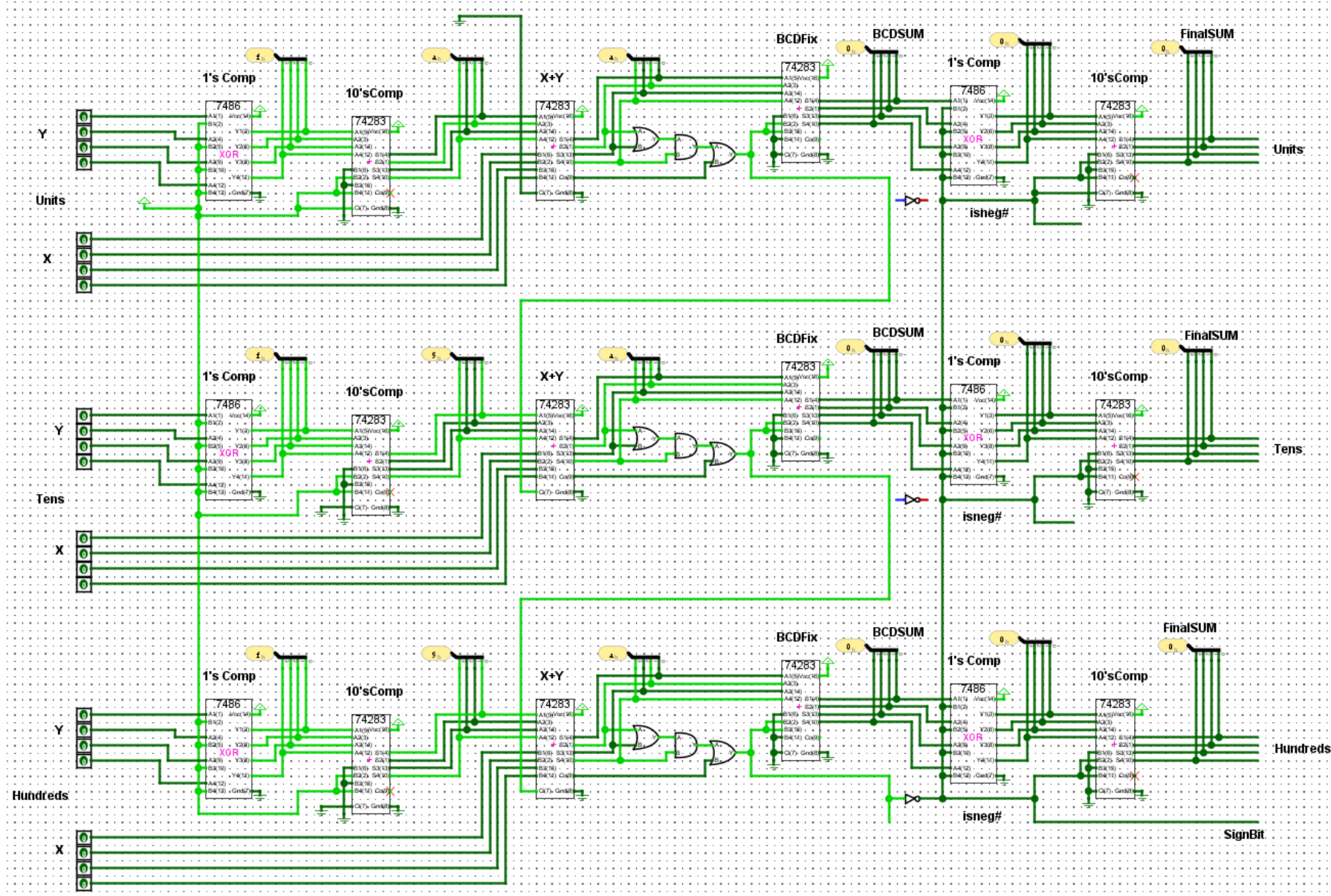
		Co		Ci	Co		Ci			
A	28		0010			1000		minuend		
B	-41		-0100			-0001		subtrahend		
	-13									
			1011			1110		1's complement of subtrahend		
			+1010			+1010	1	add 10 + ci		
		1	0101	1	1	1001		10's complement of subtrahend		
			+0010		1	+1000		add minuend		
		0	1000			0001		difference		
			+0000			+0110		bcd correction		
			1000			0111				
		Final carry=0 difference is neg (A<B)								
			0111			1000		1's complement		
			+1010			+1010		add 10		
			0001		1	0010		9's complement		
			+0000			+0000	1	add 1		
			0001			0011				

Op1-Op2	Co	A + 10#(B)	SUM
-9	0	1	-9
-8	0	2	-8
-7	0	3	-7
-6	0	4	-6
-5	0	5	-5
-4	0	6	-4
-3	0	7	-3
-2	0	8	-2
-1	0	9	-1
0	0	A	0
1	0	B	1
2	0	C	2
3	0	D	3
4	0	E	4
5	0	F	5
6	1	0	6
7	1	1	7
8	1	2	8
9	1	3	9

A single digit BCD subtractor conceptually looks like this



Example of Combinational Three-Digit Subtractor



DexpCnt For Add/Sub Overflow and Insignificant Addend (IA)

		pos/pos								neg/pos							
Xreg(Op2) →		10000000	1000000	100000	10000	1000	100	10	1	0	0	0	0.0001	0.00001	0.000001	0.0000001	
Yreg(Op1) ↓		7	6	5	4	3	2	1	0	-1	-2	-3	-4	-5	-6	-7	
10000000	7	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	
1000000	6	-1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
100000	5	-2	-1	0	1	2	3	4	5	6	7	8	9	10	11	12	
10000	4	-3	-2	-1	0	1	2	3	4	5	6	7	8	9	10	11	
1000	3	-4	-3	-2	-1	0	1	2	3	4	5	6	7	8	9	10	
100	2	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7	8	9	
10	1	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7	8	
1	0	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7	
0.1	-1	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	
0.01	-2	-9	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	
0.001	-3	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	
0.0001	-4	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	
0.00001	-5	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	
0.000001	-6	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	0	1	
0.0000001	-7	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	0	

IA

IA

Must detect that the insignificant Addend condition boundary crossing during second operand entry and set a bit.
Then at the start of and add, the exponent of the smallest number has to be backed out of DexpCnt and replaced with 0.

Overflow

Overflow happens when Sum generates a carryO of MSD.

Final DexpCnt

-14	0010
-13	0011
-12	0100
-11	0101
-10	0110
-9	0111
-8	1000
-7	1001
-6	1010
-5	1011
-4	1100
-3	1101
-2	1110
-1	1111
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110

IA boundary



DexpCnt Direction



DexpCnt Direction

IA boundary

Multiplication and Division Overflows and Underflows

OP2																														
A div C...	0.0000001 -7	0.0000001 -7	0.000001 -6	0.000001 -6	0.00001 -5	0.00001 -5	0.0001 -4	0.0001 -4	0 -3	0 -3	0.01 -2	0.01 -2	0.1 -1	0.1 -1	1 0	1 0	10 1	10 1	100 2	100 2	1000 3	1000 3	10000 4	10000 4	100000 5	100000 5	1000000 6	1000000 6	10000000 7	10000000 7
	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp	Dexp	Sexp
0.0000001 -7	0	-14	-1	-13	-2	-12	-3	-11	-4	-10	-5	-9	-6	-8	-7	-7	-8	-6	-9	-5	-10	-4	-11	-3	-12	-2	-13	-1	-14	0
0.000001 -6	1	-13	0	-12	-1	-11	-2	-10	-3	-9	-4	-8	-5	-7	-6	-6	-7	-5	-8	-4	-9	-3	-10	-2	-11	-1	-12	0	-13	1
0.00001 -5	2	-12	1	-11	0	-10	-1	-9	-2	-8	-3	-7	-4	-6	-5	-5	-6	-4	-7	-3	-8	-2	-9	-1	-10	0	-11	1	-12	2
0.0001 -4	3	-11	2	-10	1	-9	0	-8	-1	-7	-2	-6	-3	-5	-4	-4	-5	-3	-6	-2	-7	-1	-8	0	-9	1	-10	2	-11	3
0.001 -3	4	-10	3	-9	2	-8	1	-7	0	-6	-1	-5	-2	-4	-3	-3	-4	-2	-5	-1	-6	0	-7	1	-8	2	-9	3	-10	4
0.01 -2	5	-9	4	-8	3	-7	2	-6	1	-5	0	-4	-1	-3	-2	-2	-3	-1	-4	0	-5	1	-6	2	-7	3	-8	4	-9	5
0.1 -1	6	-8	5	-7	4	-6	3	-5	2	-4	1	-3	0	-2	-1	-1	-2	0	-3	1	-4	2	-5	3	-6	4	-7	5	-8	6
OP1 1 0	7	-7	6	-6	5	-5	4	-4	3	-3	2	-2	1	-1	0	0	-1	1	-2	2	-3	3	-4	4	-5	5	-6	6	-7	7
10 1	8	-6	7	-5	6	-4	5	-3	4	-2	3	-1	2	0	1	1	0	2	-1	3	-2	4	-3	5	-4	6	-5	7	-6	8
100 2	9	-5	8	-4	7	-3	6	-2	5	-1	4	0	3	1	2	2	1	3	0	4	-1	5	-2	6	-3	7	-4	8	-5	9
1000 3	10	-4	9	-3	8	-2	7	-1	6	0	5	1	4	2	3	3	2	4	1	5	0	6	-1	7	-2	8	-3	9	-4	10
10000 4	11	-3	10	-2	9	-1	8	0	7	1	6	2	5	3	4	4	3	5	2	6	1	7	0	8	-1	9	-2	10	-3	11
100000 5	12	-2	11	-1	10	0	9	1	8	2	7	3	6	4	5	5	4	6	3	7	2	8	1	9	0	10	-1	11	-2	12
1000000 6	13	-1	12	0	11	1	10	2	9	3	8	4	7	5	6	6	5	7	4	8	3	9	2	10	1	11	0	12	-1	13
10000000 7	14	0	13	1	12	2	11	3	10	4	9	5	8	6	7	7	6	8	5	9	4	10	3	11	2	12	1	13	0	14

Dexp and Sexp Counter Values From All Combinations of Op1 and Op2

Division Overflows
Anytime Dexp is positive number greater than 7
Within the positive exp1, negative exp2 quadrant

$(DPOvrfl=1)(SignYexp=0)(SignXexp=1)$

Mult Overflows
Anytime Sexp is positive number greater than 7
Within the positive exp1, positive exp2 quadrant

$(DPOvrfl=1)(SignYexp=SignXexp=0)$

○

Div Underflows

$(DPOvrfl=1)(SignYexp=1)(SignXexp=0)$

Mult Underflows

$(DPOvrfl=1)(SignYexp=SignXexp=1)$